



The EDGE Website Owner's Manual

How it works, what you are responsible for,
and how to keep it running without help.

Project	EDGE Esports Club — Ramaiah University of Applied Sciences
Live at	potatoesareathing.github.io/edge-website/
Repository	github.com/potatoesareathing/edge-website
Written	17 Sept 2026 · revised 26 Sept 2026
Covers	26 files · ~4,400 lines · zero dependencies

How to read this manual

You built a working website without knowing what most of it was called. That is a real achievement and also a real problem, because you cannot maintain what you cannot name. This manual gives you the names.

It is written from your actual code — every file opened and read, nothing assumed because it is common. Where something could not be determined from the project files, it is marked **Needs verification** rather than guessed at.

The three ways to use this

1. **Read it front to back, once.** Parts 1–7 build a mental model in order: what the thing is, what the words mean, what happens when someone visits, then each system in depth. Skipping ahead will cost you more time than reading will.
2. **Keep it beside you as a reference.** Part 9 is a maintenance inventory — every item you own, how you know it needs attention, and what breaks if you get it wrong. Part 11 is a playbook of step-by-step procedures for the jobs you will actually do.
3. **Reach for Part 12 when something breaks.** It teaches you to locate a fault before you try to fix it, which is the single skill that separates people who can maintain software from people who can only ask for help.

A PROMISE ABOUT COMPLEXITY

This manual does not simplify things to make you feel capable. Where your site does something genuinely intricate — the WebGL portal is the clearest case — you get the real explanation, including the mathematics, because a simplified version would leave you unable to debug it. Every technical term is defined the first time it appears, in terms of your code, not in the abstract.

ONE THING YOU SHOULD KNOW IMMEDIATELY

Your registration form does not work. It looks like it works. A student fills it in, sees a success screen, and their details are written to storage inside their own phone's browser, where you can never reach them. Nothing is sent anywhere.

This is not a bug in the code — the code is waiting for two configuration values you have not supplied yet. Part 6 explains the mechanism and Part 11.14 has the fix. Do not run a club fair before reading it.

Conventions

- `Monospaced text` is something literal: a filename, a command, a value in your code.
- **Bordered boxes** flag definitions, warnings, and things needing verification.

- File references look like `js/app.js:554` — that is the file, then the line number. Line numbers are accurate at commit `1ca0a30` and will drift as you edit.

What is in this manual

1. What you actually built

The architecture in one picture
The five honest sentences
What your site is not

2. The vocabulary

Client and server, frontend and backend
Dependency, build, deployment, hosting
API, endpoint, database, environment variable

3. What happens when someone opens the site

The load sequence, step by step
Script order and why it matters
The global objects

4. Read the codebase

Every file and what it owns
Safe to edit, careful, do not touch
How the files find each other

5. The frontend systems in depth

Tab routing · Content and renderers · The design system
The homeworld video · The animation layer · Sound

6. The portal, the chatbot and the form

WebGL from first principles
The FAQ matcher, honestly described
The join form and Supabase

7. Hosting, deployment and the outside world

Git, GitHub, GitHub Pages
Every external service you depend on
HTTPS, DNS, caching

8. What happens if Claude disappears tomorrow

9. The maintenance inventory

10. Known gaps and unfinished work

11. The maintenance playbook — 18 procedures

12. Troubleshooting: how to find a fault yourself

13. Things to understand before your next website

14. What to learn next — a roadmap

15. Running this site without a subscription

Everything you must manage, by how often

The two scripts that replace judgement

Tool options compared · What to watch out for

16. Checklists

17. The EDGE cheat sheet

What you actually built

Before any detail, get the shape of the thing in your head. Your site is simpler than you probably fear and more unusual than you probably realise.

1.1 The five honest sentences

1. **Your website is 22 files.** One HTML file, one stylesheet, six JavaScript files, four media files, four documents, and a few configuration files. That is all of it.
2. **It has no dependencies.** There is no `package.json`, no `node_modules` folder, no framework. Nothing was downloaded from anyone to make it run. This is rare and it is a large part of why it will still work in five years.
3. **It has no build step.** The files you edit are the exact files a visitor's browser receives. Nothing compiles, bundles, transpiles or minifies. What you see is what ships.
4. **It has no backend of its own.** No server of yours runs any code. The site is static files handed out by GitHub. The one piece of server-side work it wants to do — storing registrations — is delegated to an outside service that is not yet connected.
5. **All of its content lives in one file.** `js/content.js` holds every piece of text, every roster, the leaderboard, the events, the FAQ answers. You can run the club's website for a year without opening any other file.

1.2 The architecture in one picture

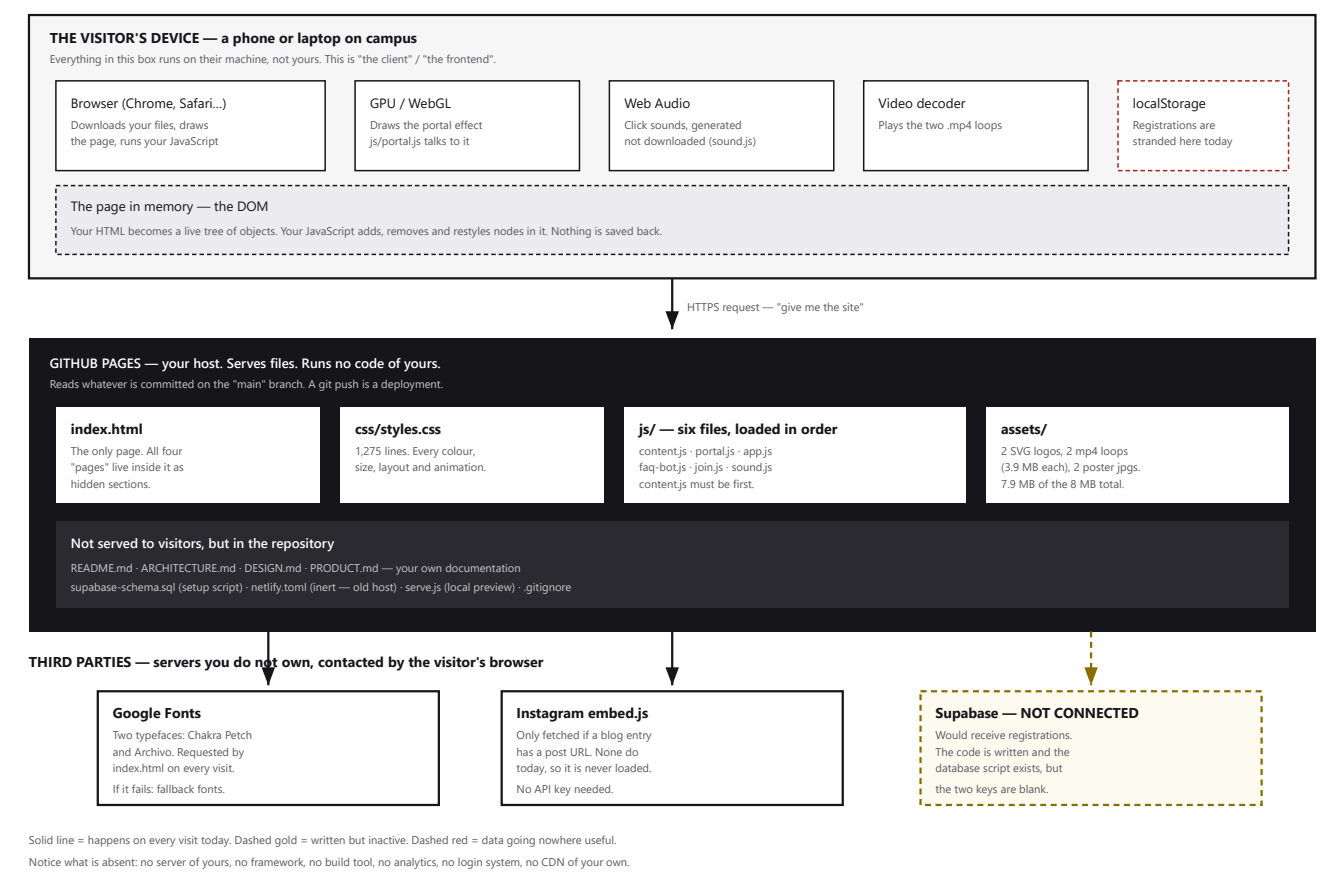


Figure 1. The whole system. Three layers: the visitor's device, the host that hands out files, and three outside services. You own the middle layer only.

1.3 What your site is *not*

Half of understanding a system is knowing what it does not contain. People will assume your site has these things because most sites do. It does not:

Thing it does not have	What that means	Why it is fine (or not)
A framework (React, Vue, Next.js)	No library imposes structure on your code. You wrote plain JavaScript that talks to the browser directly.	Fine, and a real advantage. Frameworks break as they upgrade. Plain JavaScript from 2015 still runs today. Nothing here will rot.
A build step	Nothing turns your source into different output files. Edit a file, push it, it is live exactly as written.	Fine. You lose automatic minification (smaller files) and cannot use newer syntax that old browsers reject. Neither matters at this size.
A backend	No code of yours runs on a server. Nothing can happen on your site while nobody is looking at it.	Fine for now. The moment you need email notifications, an admin page, or scheduled jobs, you will need one. Part 13 covers what that would take.
Accounts or logins	No <i>authentication</i> — no way for anyone to prove who they are. Nobody signs in, so there is nothing to sign into.	Fine, and a relief. Authentication is where most beginner sites get breached. You cannot leak passwords you never collected.

A database you run	Rosters, leaderboard and events are hardcoded in <code>js/content.js</code> and change only when you edit and push.	Fine at club scale. Editing a file and pushing takes two minutes. A database would be worth it at hundreds of rows changing weekly.
Analytics	Nothing counts visitors. You have no idea how many people use the site.	A genuine gap , and a privacy win. Part 9 covers adding a privacy-respecting counter if you want one.
Error monitoring	If the site breaks for a visitor, nothing tells you. You only find out if somebody complains.	A genuine gap. The most likely failure — the portal on an old phone — is already handled gracefully in code, but you would not know.
A custom domain	The address is <code>potatoesareathing.github.io/edge-website/</code> . You do not own a domain name, so there is no <i>DNS</i> to configure.	Fine, but weak for a club. Part 11.17 covers buying <code>edgeruas.in</code> or similar and pointing it here.
A favicon	No little icon in the browser tab. There is no <code>favicon.ico</code> in the project.	Small but visible gap. Five-minute fix, Part 11.18.
robots.txt / sitemap.xml	No instructions for search engines. Google will still find and index the site; it just has no guidance.	Minor. Worth adding once the content is real.

THE THING TO TAKE FROM THIS

You did not build a small version of a complicated thing. You built a *genuinely simple* thing that happens to have two sophisticated pieces in it (the WebGL portal and the video system). Simple systems are maintainable by one person. Keep it that way: every dependency you add is a thing that can break while you are not looking.

The vocabulary

These are the words people will use when they talk to you about your site. Each one is defined here in terms of your actual project, not in the abstract. Read this part once; you will come back to it.

2.1 Client and server

CLIENT

The machine asking for something. On your site, the client is the visitor's phone or laptop, specifically the browser running on it. Everything in the top box of Figure 1 is the client.

SERVER

A machine that sits waiting for requests and answers them. Your server is a GitHub computer somewhere. When someone types your address, their browser asks that machine for `index.html`, and it sends the file back.

Your server is a *static* server: it only hands over files that already exist. It never thinks. It cannot check a password, save a form, or send an email.

The word "server" confuses beginners because it means both the physical machine and the program running on it. When someone says "is your server down?", they mean "is the thing that hands out your files responding?" For you, that is GitHub's problem, not yours.

2.2 Frontend and backend

FRONTEND

Everything that runs on the client. Your HTML, your CSS, all six JavaScript files, the WebGL portal, the chatbot, the form validation. **Your site is 100% frontend.**

BACKEND

Code that runs on a server, where the visitor cannot see or change it. You have **no backend of your own**. Supabase, once connected, acts as a rented backend: their servers run the database, and you never manage a machine.

This distinction matters enormously for security. Anything in your frontend is readable by anyone — right-click, "View Source", and every line of your JavaScript is there. This is not a flaw; it is how the web works. It is why you must never put a secret password in frontend code, and why the FAQ chatbot is a keyword matcher rather than a real AI model (Part 6.2 explains).

2.3 How a page is actually made

HTML

The structure and content. Your `index.html` says "there is a header here, a table there, a form below it". It is a skeleton, not a picture.

CSS

The appearance. Your `css/styles.css` says "headers are white on near-black, 33 pixels tall, in Chakra Petch". Same skeleton, different clothes.

JAVASCRIPT

The behaviour. Your six `js/` files make things happen: switch tabs, answer questions, validate the form, draw the portal. HTML and CSS are descriptions; JavaScript is instructions.

THE DOM (DOCUMENT OBJECT MODEL)

When the browser reads your HTML, it builds a live tree of objects in memory. That tree is the DOM. Your JavaScript does not edit `index.html` — it edits the DOM, and the screen updates to match.

This is why your changes vanish on refresh: the file on disk never changed. It is also why `js/content.js` exists — to make content changes permanent, they have to live in a file.

2.4 Words about code organisation

COMPONENT

A reusable piece of interface. Frameworks make this formal; your site does it with *functions that build DOM nodes*. `postCard()` in `js/app.js:121` takes one blog entry and returns a finished card. That is a component, even without the label.

STATE

Information that changes while the page is open and that the interface must reflect. Your site has very little, which is why it is easy to reason about. Examples: which tab is showing (`showTab`), which join path is selected (`let path = "member"` in `js/join.js:27`), how far the portal has opened (`progress` in `js/portal.js:137`).

Bugs in complicated apps are usually state bugs: two places disagree about what is true. You avoided most of these by keeping state in one place per system.

ROUTING

Deciding what to show based on the address. Your site uses *hash routing*: the part of the URL after the `#`. Visiting `.../#players` shows the Players section. `showTab()` in `js/app.js:47` is your entire router — 20 lines.

MIDDLEWARE

Code that sits between a request arriving and it being answered, doing something to it on the way — checking a login, logging the request. **Your site has none**, because it has no backend. The word is in this manual only so you are not thrown when someone uses it.

2.5 Words about data moving

API (APPLICATION PROGRAMMING INTERFACE)

An agreed way for one program to ask another for something. Not a thing you can point at — a contract.

Your site uses three: the *browser APIs* built into every browser (WebGL, Web Audio, localStorage, fetch); the *Supabase REST API*, which is how your form would save a registration; and Instagram's embed script, which is an API of sorts.

ENDPOINT

One specific address within an API. Your form's endpoint, once configured, is

```
https://YOUR-PROJECT.supabase.co/rest/v1/registrations
```

Built in `js/join.js:211` from the URL and table name in `content.js`. Sending a `POST` request to that address means "add a row to the registrations table".

REQUEST AND RESPONSE · GET AND POST

A request asks; a response answers. **GET** means "give me something" — every file your site loads is a GET. **POST** means "here is some data, do something with it" — your form uses POST. The difference matters: GETs can be cached and repeated safely, POSTs change something and should not be repeated blindly.

STATUS CODE

A number in every response saying how it went. **200** fine. **404** no such file. **403** forbidden — this is what Netlify returned when its credits ran out. **409** conflict — your form treats this as "this register number already signed up" (`js/join.js:226`). **500** the server broke.

JSON

A text format for structured data, and the way your form talks to Supabase. It looks like JavaScript objects because it is derived from them:

```
{"path": "member", "full_name": "Asha K", "phone": "9876543210"}
```

2.6 Words about running and shipping

DEPENDENCY

Someone else's code your project needs in order to work. **Your project has zero.** Nothing is installed; nothing can break when its author changes it.

You do use three outside *services* at runtime (Google Fonts, Instagram, Supabase), which is a different kind of reliance — covered in Part 7.3.

BUILD PROCESS

A program that transforms your source files into the files that actually ship — combining, shrinking, converting newer syntax to older. **Your site has none.** `netlify.toml` says `command = ""`, meaning "there is nothing to run".

DEPLOYMENT

Getting your files from your computer to the machine that serves them. For you, deployment is one command: `git push`. GitHub Pages notices the new commit and republishes within about a minute.

HOSTING

Renting the machine and the address. GitHub Pages hosts you for free, with no bandwidth bill and no credit system — which is exactly why the site moved there.

DNS (DOMAIN NAME SYSTEM)

The phone book of the internet: it turns a name into the numeric address of a machine. **You do not use DNS**, because you do not own a domain name. GitHub owns `github.io` and manages its DNS. If you ever buy `edgeruas.in`, configuring DNS becomes your job — Part 11.17.

CDN (CONTENT DELIVERY NETWORK)

Copies of your files kept on machines around the world, so a visitor is served from one physically near them. **You get one free** — GitHub Pages runs your site through a CDN automatically. Google Fonts is also a CDN. Nothing to configure.

CACHE

A saved copy kept to avoid fetching again. Caching is the single most common cause of "I fixed it but it still looks broken". Your logo problem was exactly this: the old host was told to keep asset files for 24 hours, so a corrected logo could not reach anyone.

The fix used in your code is *cache busting* — the `?v=2` on every `edge-mark.svg?v=2` in `index.html`. Change the number and browsers treat it as a different file.

ENVIRONMENT VARIABLE

A value supplied to a program from outside its code, so the same code can behave differently in different places — and so secrets stay out of the source.

Your site has none, and cannot have any. There is no build step and no server, so there is nowhere for a value to be injected. This is why your Supabase key would sit in plain sight in `content.js`, and why the database's security rules — not secrecy — are what protect student data. Part 6.3 covers this properly; it is the most important security idea in your project.

AUTHENTICATION VS AUTHORISATION

Authentication is proving who you are (logging in). *Authorisation* is what you are allowed to do once known. Your site has no authentication. It does have authorisation, in the database: anonymous visitors may insert a registration and may do nothing else.

WEBHOOK

A reverse API call: instead of you asking a service for news, the service calls *you* when something happens. **You have none**, and could not receive one without a backend. Mentioned because it will come up if you ever add "email me when someone registers".

HTTPS AND CERTIFICATES

The `s` means the connection is encrypted, so nobody between the visitor and the server can read or alter it. It requires a *certificate* proving the site is who it claims. GitHub Pages issues and renews yours automatically, and your repository has `https_enforced: true`, so plain HTTP is redirected. **Nothing to maintain** — but if you add a custom domain you must re-enable it.

2.7 Two words specific to your site

WEBGL

A browser API for drawing with the graphics chip (GPU) instead of the normal page-drawing machinery. You reach for it when you need something per-pixel and fast — your portal's torn, refracting edge is computed for every pixel, sixty times a second. The normal CSS engine cannot do that.

Programs that run on the GPU are called *shaders* and are written in a separate C-like language, GLSL. Yours is 80 lines of text inside `js/portal.js:50-128`.

THREE.JS

A popular library that wraps WebGL in friendlier concepts — scenes, cameras, meshes. **Your site does not use it**, deliberately. Three.js is around 600 KB; your portal has no scene, no camera and no 3D geometry, so it would be a large download to do less. Knowing the name matters because people will assume it is there.

What happens when someone opens the site

Follow one visitor from typing the address to the page being usable. If you understand this sequence, most "why is it doing that?" questions answer themselves.

3.1 The sequence

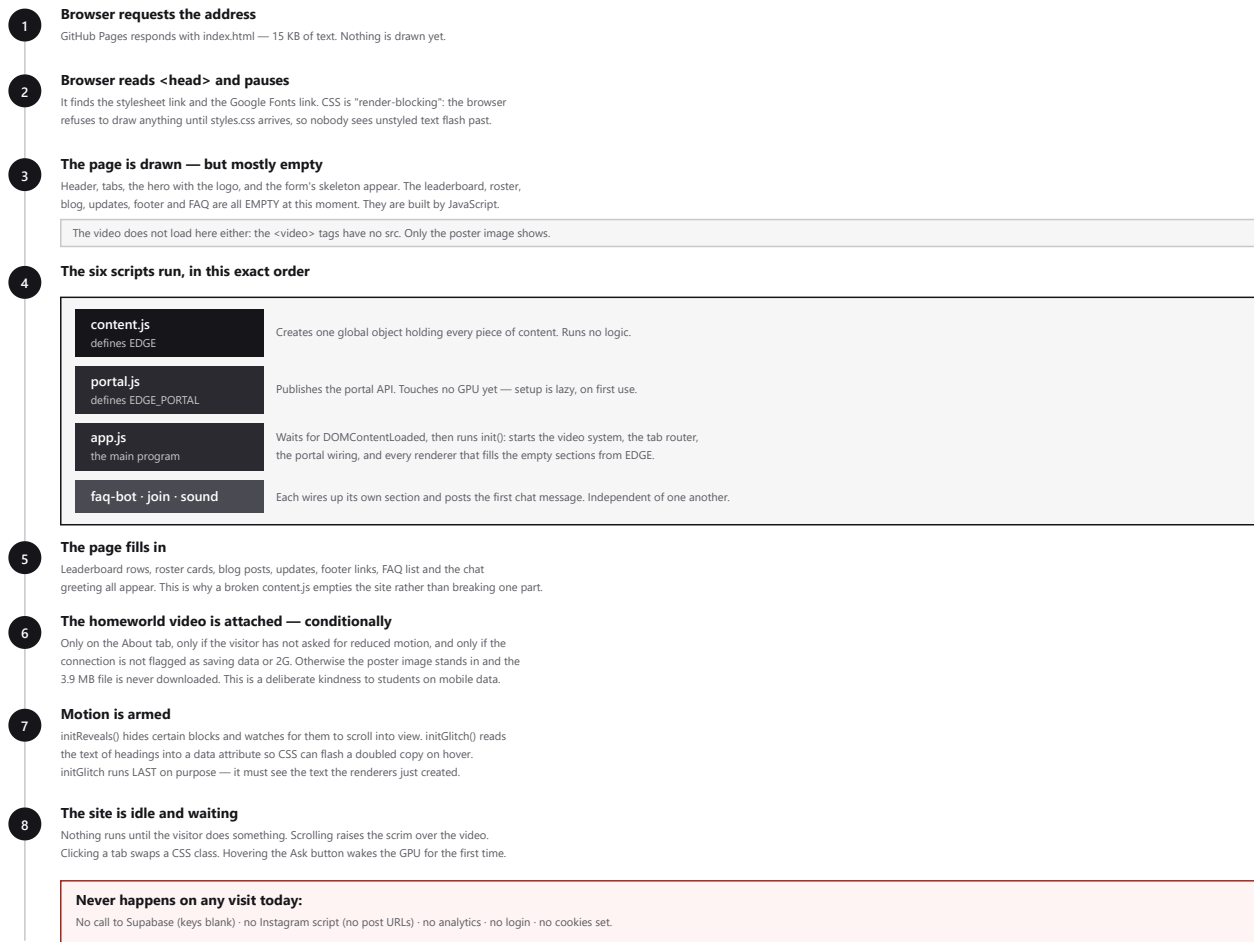


Figure 2. From address bar to idle page. Steps 1–3 are the browser; 4–7 are your code; step 8 is the resting state.

3.2 Why the script order is not negotiable

At the bottom of `index.html`:

```
<script src="js/content.js"></script>
<script src="js/portal.js"></script>
<script src="js/app.js"></script>
<script src="js/faq-bot.js"></script>
<script src="js/join.js"></script>
<script src="js/sound.js"></script>
```

Plain `<script>` tags run strictly in order, each finishing before the next starts. That is the entire mechanism holding your site together, and it imposes two rules:

- **content.js must come first.** It creates the global `EDGE` object. Every other file reads from it. Move it later and four files fail at once — `app.js:14` would log `[EDGE] content.js did not load.` and stop.
- **portal.js must come before app.js.** `app.js` reads `window.EDGE_PORTAL` when it wires up the Ask button.

The last three are independent of each other and could be reordered, though there is no reason to.

WHY THE SCRIPTS ARE AT THE BOTTOM OF THE BODY

A script tag in the `<head>` blocks the browser from drawing until it has downloaded and run. Putting them last means the visitor sees the page before the JavaScript arrives. Combined with `DOMContentLoaded`, it also guarantees the HTML elements exist by the time your code looks for them.

3.3 The IIFE pattern, and why every file is wrapped

Every one of your JavaScript files looks like this:

```
(function () {  
  "use strict";  
  // ... everything  
})();
```

IIFE — IMMEDIATELY INVOKED FUNCTION EXPRESSION

A function that is defined and run in the same breath. Anything declared inside is invisible outside. Without it, every variable in every file would share one global namespace, and two files using `const log` would collide and break each other.

`"use strict"` turns silent mistakes into loud errors — assigning to an undeclared variable throws instead of quietly creating a global.

Because the files are sealed, anything they deliberately share is attached to `window` — the browser's global object. Your site has exactly five such handoffs:

Global	Created by	Used by	What it is
<code>EDGE</code>	<code>content.js</code>	<code>app</code> , <code>faq-bot</code> , <code>join</code>	The entire content of the site as one object. The most important global.
<code>EDGE_PORTAL</code>	<code>portal.js</code>	<code>app.js</code>	The portal's controls: <code>configure</code> , <code>peek</code> , <code>open</code> , <code>close</code> , <code>suspend</code> , <code>isOpen</code> , <code>supported</code> .
<code>EDGE_ASK</code>	<code>faq-bot.js</code>	nothing yet	Lets other code push a question into the chat. Written for a future deep-link from the Join tab.

EDGE_SOUND	sound.js	nothing yet	play() , mute() . Exposed so you could add a mute toggle later.
__edgeHomeworld	app.js	app.js	Internal. Lets the tab router tell the video system which tab is showing. The double underscore is a convention meaning "private, do not rely on this".

3.4 How data flows through the site

This is the pattern worth internalising, because it is the one you will use whenever you change content.

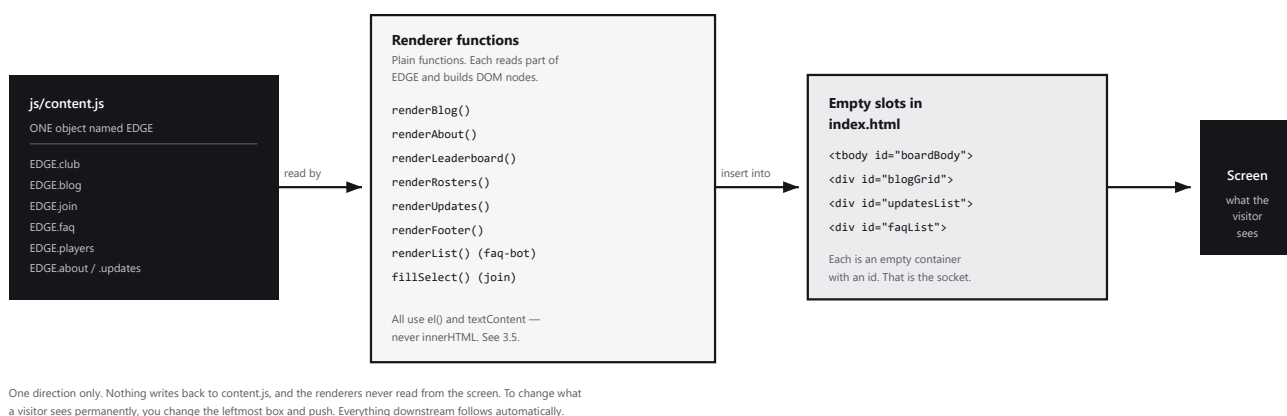


Figure 3. The content pipeline. This one-way flow is the reason you can maintain the site by editing a single file.

3.5 One security decision worth understanding

Every renderer builds text through a helper at `js/app.js:20`:

```
function el(tag, className, text) {
  const n = document.createElement(tag);
  if (className) n.className = className;
  if (text !== null) n.textContent = text;    // <== the important line
  return n;
}
```

Setting `textContent` puts text in as *text*. The alternative, `innerHTML`, puts it in as *markup* — meaning any HTML inside the string becomes real elements.

WHY THIS MATTERS — CROSS-SITE SCRIPTING

Suppose a player's in-game name were `<script>steal()</script>`. With `textContent`, visitors see those characters printed on screen. With `innerHTML`, the browser would *run* it, in every visitor's session. That attack is called **XSS (cross-site scripting)** and it is one of the most common vulnerabilities on the web.

Today all your content is written by you, so the risk is low. The moment any text comes from a database or a form, it becomes real. **The rule: never introduce `innerHTML` into this project.** There is currently not a single use of it anywhere in your JavaScript, and that is worth keeping true.

Read the codebase

A tour of every file you own: what it is responsible for, whether it is safe to edit, and how it connects to the others. This is the part to have open the first few times you change something.

4.1 The whole project, annotated

EDGE-Website/		
├─ index.html	354 lines	The only page. All four tabs live inside it.
├─ css/		
│ └─ styles.css	1,275 lines	Every visual decision, in 20 numbered sections.
├─ js/		
│ └─ content.js	334 lines	ALL CONTENT. Edit this one.
│ └─ app.js	862 lines	Router, renderers, video, motion, portal wiring.
│ └─ portal.js	428 lines	The WebGL transition. Self-contained.
│ └─ faq-bot.js	278 lines	The chatbot matcher.
│ └─ join.js	374 lines	Form validation and submission.
│ └─ sound.js	91 lines	Synthesised click sounds.
├─ assets/		
│ └─ edge-mark.svg		The ghost. 398 x 333. Traced from your artwork.
│ └─ edge-wordmark.svg		The EDGE lettering. 665 x 183.
│ └─ video/		
│ └─ homeworld.mp4	3.9 MB	Homepage loop, 7s, crossfaded, no audio.
│ └─ homeworld-poster.jpg	100 KB	Still frame shown before/instead of the video.
│ └─ faq-world.mp4	3.9 MB	The pixel world. 7s, crossfaded, HAS audio.
│ └─ faq-world-poster.jpg	120 KB	
├─ README.md		Setup, deployment, how to change things.
├─ ARCHITECTURE.md		The 13 invariants – rules that must stay true.
├─ DESIGN.md		Why the site looks as it does. Records past mistakes.
├─ PRODUCT.md		What the club is and who the site is for.
├─ supabase-schema.sql		Run once in Supabase to create the database.
├─ serve.js	33 lines	Tiny local preview server. Never deployed usefully.
├─ netlify.toml	27 lines	Old host's config. INERT on GitHub Pages.
└─ .gitignore		What git must never publish.

4.2 Safe, careful, and dangerous

Judge every file by one question: *if I get this wrong, how much breaks and how obviously?*

File	Verdict	Why
js/content.js	Safe	Designed to be edited. It is data, not logic. The worst realistic mistake is a missing comma or quote, which empties the site — loudly and obviously, and fixed by undoing your change. Make 90% of your edits here.
The four .md docs	Safe	Not served to visitors. Cannot break anything. Keep them current; they are your memory.

<code>index.html</code>	Careful	Safe to change visible words. Dangerous to change or remove an <code>id</code> — JavaScript finds elements by id, and a renamed id means a renderer silently does nothing. Also do not reorder the script tags (3.2).
<code>css/styles.css</code>	Careful	Can only break appearance, never logic — but appearance breaks can make the site unusable (an invisible button is a broken button). The specificity trap in 4.6 has bitten this project twice.
<code>js/app.js</code>	Careful	The largest logic file. A syntax error here stops the router, every renderer, the video and the portal wiring at once. Change one function at a time and reload.
<code>js/join.js</code>	Careful	Handles real student data. A broken validation rule means bad data or a form nobody can submit. Always test both paths (Member and Roster) after touching it.
<code>js/faq-bot.js</code>	Careful	The matching logic is tuned. You almost never need to edit this file — to change what the bot says, edit <code>content.js</code> . See 6.2 before touching the scoring.
<code>js/portal.js</code>	Dangerous	Contains a GPU shader in a different language (GLSL), embedded as a string. A typo there does not crash — it prints a warning to a console you are not watching and the portal silently falls back to no effect. Do not edit without reading 6.1.
<code>js/sound.js</code>	Careful	Small and isolated. Worst case: a bad number makes a loud or ugly noise. Easy to revert.
<code>.gitignore</code>	Dangerous	Not because it breaks the site — because removing lines can <i>publish things that must stay private</i> . Its first three lines keep third-party video out of your public repository. See 4.5.
<code>netlify.toml</code>	Obsolete	Does nothing now. Keep it or delete it; see 4.4.
<code>serve.js</code>	Safe	Development only. Even if you broke it completely the live site is unaffected.
<code>assets/*</code>	Careful	Replacing a file is fine. Changing an image's aspect ratio is not — the logo crisis came from a square file replacing a 398×333 one. See 11.9.

4.3 What each file is responsible for

index.html — the skeleton and the sockets

Three jobs. First, it declares the page's metadata: the title, the description search engines show, the `viewport` tag that makes the site work on phones at all. Second, it lays out the visible structure. Third, and most importantly for you, it provides **empty containers with ids** for JavaScript to fill.

Its structure, top to bottom:

- `.homeworld` — the background video layer, deliberately placed outside everything else. The comment at line 16 explains why: the panel entrance animation uses `clip-path`, and a clip-path on a parent also clips fixed-position children. Nesting the video inside would have cropped it.
- `.topbar` — logo and the four tab buttons.
- `main.shell` — four `<section class="panel">` elements, one per tab. Only one has the class `is-active` at a time.
- `.footer`

- `#portalCanvas` , `#faqVideo` , `.ask-fab` , `#faqWorld` — the Ask EDGE system, all outside `main` because it is not part of any tab.
- The six script tags.

THE IDS JAVASCRIPT DEPENDS ON

Renaming or deleting any of these silently disables a feature. There is no error; the section just stays empty.

`homeworld` · `homeworldVideo` · `homeworldScrim` · `panel-about` · `panel-players` · `panel-updates` · `panel-join` · `tab-about` (and the other three) · `aboutLead` · `aboutBody` · `aboutFacts` · `aboutPillars` · `aboutCrew` · `aboutCrewWrap` · `blogFilters` · `blogLead` · `blogGrid` · `boardBody` · `boardNote` · `viewLeaderboard` · `viewRoster` · `countdownSlot` · `updatesList` · `updatesArchive` · `joinForm` · `joinHeading` · `joinIntro` · `pathValue` · `memberFields` · `rosterFields` · `interestChecks` · `formStatus` · `joinSuccess` · `footerLinks` · `footerTagline` · `footerUni` · `year` · `portalCanvas` · `faqVideo` · `askFab` · `faqWorld` · `worldBack` · `chatLog` · `chatChips` · `chatForm` · `chatInput` · `faqList`

js/content.js — everything you will actually change

One object, `EDGE` , with seven top-level sections:

Key	Line	Holds
<code>club</code>	15	Name, university, tagline, email, and the three social links (Instagram, Discord, YouTube). All three social links are currently empty strings , so the footer renders no links at all.
<code>blog</code>	38	The feed on the About tab. Two entry types: <code>"post"</code> (your own photo or clip) and <code>"instagram"</code> (paste a post URL, no API key needed). An Instagram entry with an empty <code>url</code> is skipped silently.
<code>join</code>	71	Form copy, the branch list, years, game titles, non-playing interests, success messages — and, at line 126, <code>backend</code> , which decides where submissions go.
<code>faq</code>	144	The chatbot's entire brain: greeting, fallback message, suggestion chips, and the <code>entries</code> array — each a question, an answer and a list of keywords.
<code>players</code>	222	<code>leaderboard</code> (the table) and <code>rosters</code> (the teams).
<code>about</code>	270	Lead paragraph, body paragraphs, the four facts, the four pillars, and <code>crew</code> — the committee. Leave <code>crew</code> empty and the whole section hides itself.
<code>updates</code>	307	Events, deadlines, announcements and results. Dates drive the countdown and the past/upcoming split automatically.

js/app.js — the main program

Read it as seven independent systems that happen to share a file. Each is separated by a banner comment.

System	Line	Responsibility
--------	------	----------------

Helpers	16–41	<code>byId</code> , <code>e1</code> , <code>fill</code> , <code>parseDate</code> . Small and used everywhere.
Tab router	43–90	<code>showTab()</code> and <code>initTabs()</code> . Also handles keyboard arrows and the URL hash.
Blog	92–237	<code>postCard()</code> , <code>renderBlog()</code> , the tag filters, and the lazy Instagram script loader.
Players	238–342	Leaderboard table, roster cards, and the Leaderboard/Roster sub-tab switch.
Updates	343–466	Event rows, the live countdown to the next deadline, past/upcoming split.
About + footer	467–543	<code>renderFooter()</code> , <code>renderAbout()</code> .
Ask wiring	544–642	<code>initAsk()</code> — when to peek, when to commit, when to come back. Owns no effect itself.
Magnet + glitch	644–718	The launcher leaning toward the cursor; the hover text flash.
Homeworld	719–799	The background video's entire lifecycle.
Reveals + boot	800–862	Scroll-triggered entrance, then <code>init()</code> which starts everything in order.

css/styles.css — twenty numbered sections

Find things by its section banners rather than by scrolling:

1. TOKENS the colour and font variables – start here
2. BASE resets, body, typography
3. PAINTED BOARD the .board and .plate components
4. NAV RAIL the top bar and tabs
5. PANELS the four tab surfaces
6. BLOG 7. JOIN 8. FAQ 9. PLAYERS 10. UPDATES
11. ABOUT 12. FOOTER
13. RESPONSIVE the first phone adjustments
14. CLASSES EMITTED BY app.js styles for nodes JavaScript creates
15. SECTION HEADINGS
16. ASK EDGE – PORTAL AND WORLD the largest section
17. INTERACTION hover, glitch, magnet
18. HOMEWORLD the video layer and scrim
19. SCROLL REVEAL
20. PHONES the max-width: 680px overrides – added last, overrides earlier rules

TOKENS: CHANGE A COLOUR IN ONE PLACE

Section 1 defines variables used everywhere else:

```
--board: #101012;    --white: #F5F5F3;    --grey: #A6A6A3;
--signal: #FFFFFF;    --display: "Chakra Petch";    --body: "Archivo";
```

Change `--board` and every surface on the site changes with it. This is why the whole design could be converted from colour to monochrome without rewriting components.

4.4 Files that are obsolete, generated, or misleading

You asked to be told about these explicitly. There are four.

File	Status	What you should know
<code>netlify.toml</code>	Inert	Written for Netlify, which no longer hosts you. GitHub Pages ignores it entirely. Its cache-control rules — including the one-week cache on videos — are not in effect. This matters: GitHub Pages sets its own cache headers and you cannot configure them. Keep the file only if you might return to Netlify; it is otherwise dead weight that will confuse you in a year.
<code>serve.js</code>	Dev only	A 33-line local web server. Useful for previewing with Instagram embeds, which do not always run from a <code>file://</code> address. It is committed and therefore published, but GitHub Pages will never execute it — Pages runs no code. Harmless.
<code>supabase-schema.sql</code>	Not yet run	A setup script, not part of the running site. Read it before you connect the database; section 3 of it is the most security-critical text in your project.
<code>_archive-v1/</code>	Local only	The first version of the site, kept on your disk and excluded by <code>.gitignore</code> . Not published, not backed up by GitHub. If you want it preserved, copy it somewhere else — a disk failure loses it.

4.5 The dangerous lines in `.gitignore`

```
assets/frames/  
assets/jinn_punch.original.mp4
```

These exclude AI-generated footage of a Tekken character and the frames cut from it. It is third-party intellectual property, it is not used by the site, and it must not be published. It was once accidentally put online, because a deploy command that uploads a folder ignores `.gitignore` entirely.

TWO RULES THAT FOLLOW FROM THAT INCIDENT

1. **Never deploy by uploading a folder.** Deploy by pushing to git. Git only publishes what is committed, so `.gitignore` is actually enforced.
2. **Deleting a line from `.gitignore` can publish a file on your next commit.** Before removing any line, look at what it is hiding.

Note also that git history is permanent: once something is committed and pushed, deleting it later does not remove it from the repository's history. Anyone can recover it. Getting this right the first time is the only reliable approach.

The file also excludes the two **video source files** (`homeworld-source.mp4`, `faq-world-source.mp4`). Those are the originals before the crossfade edit. They exist only on your computer. **They are not backed up anywhere.** If you ever need to re-edit the loops, you need them — copy them to cloud storage.

4.6 The CSS trap that has bitten this project twice

When two CSS rules target the same element, the browser picks the winner by *specificity* — a score based on how precisely the selector names the element, not on which rule appears later.

```
table.board-table { min-width: 640px; } /* element + class = stronger */
.board-table      { min-width: 0;      } /* class only      = weaker   */
```

The second rule loses even though it comes later and sits inside a phone media query. This is exactly what happened: the leaderboard stayed 640px wide on a 375px phone, and nothing in the browser reported an error. The fix was to match the weight:

```
table.board-table { min-width: 0; }
```

The same mistake in the opposite direction broke the logo. A blanket rule `img[src*="edge-mark.svg"]` was added to control the mark's size; an attribute selector outranks a plain class, so it overrode every component's own height and collapsed the logos.

THE HABIT TO BUILD

Before writing a CSS override, find the rule you are trying to beat and copy its selector shape. If the original says `table.board-table`, your override says `table.board-table`. Verify in the browser's element inspector, which shows overridden rules struck through — if your new rule appears struck through, you lost on specificity.

4.7 Where secrets should live

A hard truth for a static site: **there is nowhere truly safe**. Anything the browser can read, a visitor can read. You have no server and no build step, so there is no mechanism to hide a value.

This means:

- The Supabase *anon key* goes in `content.js` in plain sight. That is by design — it is called "anon public" because it is meant to be public.
- The Supabase *service role key* must **never** appear in this project, in any file, ever. It bypasses all security rules. It belongs only in the Supabase dashboard.
- Your GitHub password and any access tokens live in your machine's credential manager, never in a file in the repository.
- If you ever need a genuine secret in a web request — an email service key, a payment key — that request must be made by a backend, not by the browser. There is no workaround.

The frontend systems in depth

Six systems, each self-contained. Read the ones you plan to touch.

5.1 Navigation: four tabs, one page

Your site looks like it has four pages. It has one. All four sections exist in `index.html` from the moment it loads; switching tabs adds and removes a CSS class.

```
function showTab(name, push) {
  if (!TABS.includes(name)) name = TABS[0];           // unknown → About

  document.querySelectorAll(".tab").forEach(btn => {   // 1. update buttons
    btn.setAttribute("aria-selected", String(btn.dataset.tab === name));
  });
  document.querySelectorAll(".panel").forEach(p => {  // 2. show one panel
    p.classList.toggle("is-active", p.id === "panel-" + name);
  });

  if (push && location.hash !== "#" + name) {         // 3. update the URL
    history.replaceState(null, "", "#" + name);
  }
  window.scrollTo({ top: 0, behavior: "auto" });      // 4. back to the top
  if (window.__edgeHomeworld) window.__edgeHomeworld(name); // 5. tell the video
}
```

What this buys you, and what it costs

Advantage	Cost
Switching tabs is instant — no network request, nothing to load.	The first load is bigger, because all four sections arrive at once. At your size this is negligible.
No routing library, no server configuration for URLs.	Search engines see one page with one title and description. Google cannot show "EDGE — Join" as a separate result.
Sharing <code>.../#players</code> works, and the back button behaves.	Actually, it does not fully: <code>history.replaceState</code> <i>replaces</i> the current history entry rather than adding one, so the back button skips past tab changes. This is a deliberate trade — the alternative traps people who clicked four tabs behind four back presses.

ARIA AND WHY THOSE ATTRIBUTES ARE THERE

`role="tablist"`, `role="tab"`, `aria-selected`, `aria-controls` are **accessibility** markup. A screen reader announces "tab, 2 of 4, selected" instead of "button". `initTabs()` also implements left/right arrow navigation, because that is what a screen-reader user expects from anything calling itself a tablist.

If you add a fifth tab you must add all four attributes, or the site becomes harder to use for people who need it most. Procedure 11.4 has the full checklist.

5.2 The renderers

Eight functions turn `EDGE` into DOM. They share a shape worth recognising: *read data* → *build nodes* → *replace the container's children*.

```
const fill = (node, kids) => { if (node) node.replaceChildren(...kids); };
```

`replaceChildren` empties the container and inserts the new nodes in one step. This is why running a renderer twice cannot duplicate content — a real bug class in hand-written DOM code.

The pieces worth knowing about

- **Empty states are designed, not accidental.** `emptyState()` at `app.js:114` produces a lettered panel explaining what goes there. Your site launches nearly empty, so these are what most visitors will see first. They are treated as a first-class surface.
- **Dates are parsed manually** (`app.js:33`). Writing `new Date("2026-09-28")` makes JavaScript interpret it as UTC midnight, which in India (UTC+5:30) is still the 28th — but for negative offsets it lands on the 27th. Splitting the string and building the date from parts avoids the whole class of off-by-one-day bug.
- **The countdown** (`app.js:410`) ticks every second toward the nearest future deadline in `EDGE.updates`. It finds that deadline itself; you never configure it. Add an event with a future date and the countdown retargets.
- **Updates split themselves** into upcoming and past by comparing dates to today. You do not archive anything manually.
- **The blog filters build themselves** from whatever `tag` values exist in your posts. Invent a new tag and a new filter chip appears.

5.3 The design system

The site has one visual idea, stated in `DESIGN.md`: *a hand-painted signboard*. Painted boards, cream-white lettering, heavy ruled borders, and exactly one reserved primary action.

Component	Class	Rule
Board	<code>.board</code>	Replaces cards everywhere. A 3px outer rule with a thin inner rule set in from it. Depth comes from border weight and overlap — there are no soft shadows, no glass and no glow anywhere in the system.
Plate	<code>.plate</code>	The buttons. They shift 1px on press, like something physical.
Primary plate	<code>.plate--primary</code>	The only solid white fill on the page. A white fill means "this is the primary action". If you put one on something else, that is a bug, not a style choice.
Rule	<code>.rule</code>	The double line under a heading.
Painted text	<code>.painted</code>	A hard 3px offset shadow, never blurred. A soft shadow is a different craft and does not belong here.

`DESIGN.md` also carries a **Refused** list — things deliberately held out, each a regression if it returns: eyebrow labels above headings, section numbers, uniform card grids as page structure, thick coloured one-sided borders, decorative grid backgrounds, gradient text, glass, glow, near-black with a neon accent, and animating layout properties.

WHY A REFUSAL LIST IS WORTH MORE THAN A STYLE GUIDE

Style guides describe what to do and are ignored under deadline. A refusal list describes what not to do, and it is checkable: you can look at a change and ask "does this introduce something on the list?" It is how the site stayed coherent across many separate edits.

Responsive design

MEDIA QUERY

A block of CSS that applies only when a condition is true — usually a screen width. `@media (max-width: 680px) { ... }` means "apply this on screens up to 680 pixels wide".

Your site has eleven, at four breakpoints: **900px** (tablet layout), **680px** (phones — the big one, section 20), **portrait under 860px** (the FAQ world's character framing), and **landscape under 520px tall** (phone held sideways). Two more respond to `prefers-reduced-motion` rather than size.

PREFERS-REDUCED-MOTION

An operating-system setting a person turns on if animation makes them ill — vestibular disorders are common and motion can cause genuine nausea. Your site checks it in CSS and in four places in JavaScript, and when it is on: the glitch effect is removed entirely, scroll reveals are disabled, the magnet is off, the background video is never even downloaded, and the portal is skipped in favour of going straight in.

This is not decoration. It is the difference between a site someone can use and one they cannot.

5.4 The homeworld video

The looping film behind the first screen of the About tab. Three rules govern it, all about not wasting a student's mobile data.

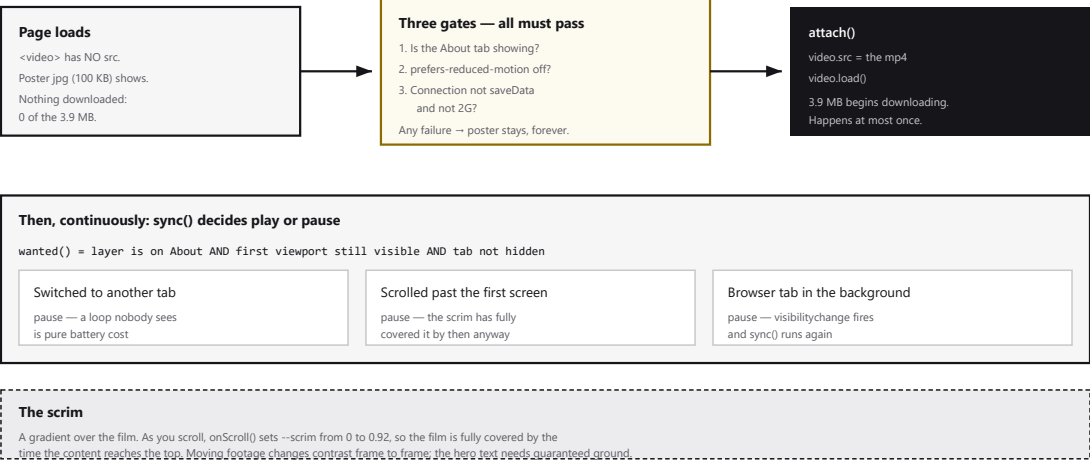


Figure 4. The video's lifecycle. Note that failure is always graceful: every path ends with a poster image rather than an empty box.

The crossfaded loop

The original cut hard at the join, producing a visible jump every 8 seconds. The shipped file is 7 seconds: its first second is a blend of the source's tail fading into its head, so the last frame and the first frame are one frame apart in the original timeline.

Measured, the junction differs by **6.6 RMS** against **9.8** for ordinary adjacent frames mid-clip — the loop point is a smaller visual step than normal motion, so there is nothing to see. The re-encode also dropped the audio track and took the file from 7.9 MB to 3.9 MB.

NEEDS VERIFICATION

The exact `ffmpeg` command used to produce the crossfade is not recorded anywhere in the project. If you need to re-cut a loop, you will have to rebuild the command. The technique is a `blend=all_expr` filter for video and `acrossfade` for audio. Procedure 11.2 includes a working command — test it before relying on it.

5.5 The animation layer

Three effects, one grammar. `DESIGN.md` is explicit that these are a deliberate system rather than scattered flourishes.

Effect	Where	How it works
Paint-on reveal	<code>app.js:805</code>	Content wipes in left-to-right, the same gesture as the panel entrance — rather than the generic fade-and-rise. Uses an IntersectionObserver : the browser tells your code when an element scrolls into view, which is far cheaper than checking positions on every scroll event. Each element is unobserved after revealing, so it never re-hides.
Hover glitch	<code>app.js:702</code>	Two offset copies of a string flash for about 0.26s. <code>initGlitch()</code> copies each element's own text into a <code>data-text</code> attribute so CSS can draw the doubled copies. It must run last in <code>init()</code> — it reads text the renderers have just created.

TWO PERFORMANCE RULES THIS CODE FOLLOWS

Only animate `transform` and `opacity`. These can be handled by the GPU without recalculating the page's layout. Animating `width`, `padding`, `height` or `margin` forces the browser to re-measure everything, every frame — that is what "jank" is. Two such animations shipped briefly in this project and were caught.

Never call `getBoundingClientRect()` in a `mousemove` handler. It forces the browser to compute layout immediately. The magnet caches the button's position and only re-measures on resize (`app.js:661`). Doing it per-event is exactly how a small flourish becomes a stuttering page.

The safety net, and the lesson behind it

```
setTimeout(() => {  
  if (!document.querySelector(".rise.is-in")) {  
    targets.forEach(node => node.classList.add("is-in")); // show everything  
  }  
}, 2000);
```

The reveal effect works by *hiding* elements until the observer says otherwise. If the observer never fires, the page stays blank — the one failure a scroll reveal must not have. After two seconds, if nothing has revealed at all, everything is shown. It only triggers when the effect is already broken.

A HARD-WON RULE: DO NOT USE `REQUESTANIMATIONFRAME` FOR CORRECTNESS

`requestAnimationFrame` (rAF) asks the browser to run code before the next frame. It is the right tool for *smoothing* animation. It is the wrong tool for anything that must simply *happen*, because **rAF does not fire in a backgrounded or throttled tab**.

This bit the project three separate times: the FAQ drawer sat parked off-screen while still focusable; the scrim never updated; the portal handover never fired. All three are now driven by direct calls or by `setInterval`. The portal's audio ramp uses a 40ms interval specifically for this reason (`portal.js:303`) — whether sound is audible at all must not depend on rAF running.

5.6 Sound

`js/sound.js` plays a dry tick on press. Nothing is downloaded — the sound is *generated* using the Web Audio API: an oscillator whose pitch falls, a volume envelope that opens in 4 milliseconds and closes over 55, and a highpass filter to keep it a tick rather than a thud.

```
const VOICES = {  
  tap: { from: 1500, to: 620, peak: 0.028, len: 0.055 }, // chips, tabs, FAQ rows  
  press: { from: 900, to: 260, peak: 0.055, len: 0.085 } // primary actions  
};
```

Which control gets which voice is decided by two CSS selectors at `sound.js:76`, matched against whatever was clicked. This is called **event delegation**: one listener on the document rather than a listener per button, so anything rendered later from `content.js` is covered without rebinding.

Browsers refuse to play audio before a visitor has interacted with the page — an anti-annoyance rule. Here that condition is met by construction, because every trigger is itself a click.

TO CHANGE THE SOUNDS

`peak` is volume (0.028 is deliberately quiet). `from` and `to` are the pitch sweep in hertz. `len` is the duration in seconds. To silence the site entirely, open the browser console and run `EDGE_SOUND.mute(true)` — or delete the `<script src="js/sound.js">` line, which removes the feature cleanly.

The portal, the chatbot and the form

The three most intricate parts of your site. The portal because it is genuinely advanced, the chatbot because people will misunderstand what it is, and the form because it is the only part handling real personal data — and the only part currently broken.

6.1 The dimensional portal

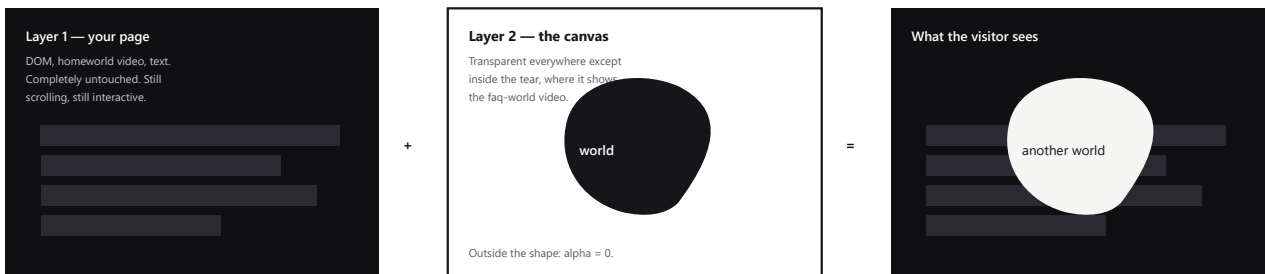
Hovering the Ask EDGE button tears an irregular opening in the page, through which a pixel-art world is visible. Clicking commits: the tear grows until that world is all there is. It reverses on the way back.

The central trick

THE CANVAS DRAWS ONLY THE DESTINATION WORLD

Everywhere outside the tear, it writes **alpha 0** — fully transparent. Your actual page is simply still there underneath, showing through.

Nothing about the homepage is captured, copied or re-rendered. That is what keeps the whole effect to a single shape and one small program. A naive implementation would render both worlds and blend them; this renders one world with a hole-shaped mask.



The tear is not a shape drawn on the page. It is a region where the canvas decides to be opaque, computed per pixel, sixty times a second, from a noise function. That is why it can be irregular, wobbling and refracting all at once — and why it needs the GPU. CSS can clip to a shape, but it cannot recompute that shape every frame per pixel.

Figure 5. Masking, not blending. One layer, one hole.

What actually runs on the GPU

SHADER

A small program that runs on the graphics chip, once per pixel, in parallel. Yours is written in GLSL and lives as a string in `portal.js:50–128`. It receives the pixel's position and a handful of values from JavaScript, and returns a colour and an opacity.

UNIFORM

A value passed from JavaScript to the shader, the same for every pixel in a frame. Yours are: `uTex` (the video frame), `uRes` (canvas size), `uTexRes` (video size), `uCenter` (where the tear opens), `uProgress` (0 closed to 1 fully open), `uMaxR` (radius needed to cover the screen), `uTime`.

`uProgress` is the one that matters. JavaScript eases it toward a target; the shader draws whatever that number says. That separation is why a hover can become a click mid-animation without anything restarting.

The shader's core, annotated:

```
float wob = fbm(vec2(cos(ang), sin(ang))) * 2.4 + uTime * 0.05);
// fbm = fractal noise. Fed the ANGLE around the centre, so the wobble
// varies around the rim rather than across the screen. uTime makes it drift.

float r = uProgress * uMaxR; // ideal radius right now
r *= 1.0 + (wob - 0.5) * 0.26 * (1.0 - uProgress * 0.8);
// Distort that radius by up to ±13%, and fade the distortion as it opens -
// so a small tear is ragged and a finished world is not permanently rippling.

float edge = 1.5 + 54.0 * (1.0 - uProgress); // soft edge, sharpens as it grows
float mask = 1.0 - smoothstep(r - edge, r + edge, dist);
if (mask <= 0.002) discard; // outside the tear: draw nothing
```

Three finishing effects, all fading out via `settle` as the tear opens:

- **Refraction** — near the rim the world is pulled toward the opening, the way something seen through a lens bends at the boundary.
- **Chromatic aberration** — red, green and blue are sampled at very slightly different positions along the radius, a fraction of a percent. This is the artefact real lenses produce; it reads as physical rather than digital.
- **Rim light** — `pow(rim, 7.0)` concentrates a thin white line right at the edge. Monochrome, no hue added, because the design system forbids colour in chrome.

The public API — and why it is reusable

You asked for a system that could later carry Players, Updates and Join into their own worlds. That is why `portal.js` knows nothing about the FAQ. It takes a video and an origin element and does its job.

```
EDGE_PORTAL.configure({ video, origin, volume }) // tell it where it leads
EDGE_PORTAL.peek(true|false) // small tear, for hover
EDGE_PORTAL.open({ video, origin, volume, onEnter }) // full transition
EDGE_PORTAL.close() // reverse it
EDGE_PORTAL.suspend() // hidden tab: stop everything
EDGE_PORTAL.isOpen() / EDGE_PORTAL.supported()
```

To add a second world later you would create another video, another destination section, and call the same four methods. Procedure 11.16 has the steps.

Five failures this code already survives

Each of these was a real bug. They are documented because the guards look like paranoia until you know why they exist.

Failure	Guard
Visitor's device has no WebGL	<code>supported()</code> returns false and <code>commit()</code> calls <code>enter()</code> directly (<code>app.js:599</code>). They go straight to the world with no effect. Nothing is broken, only plainer.
The video has not buffered yet	<code>open()</code> checks <code>readyState</code> . If there is no decodable frame it holds the peek open and waits for <code>loadeddata</code> , with an 1,800ms cap. Opening onto an empty texture is what once made the transition look like nothing happened.
The animation loop stalls	A 2,600ms timer forces the handover and snaps progress to 1. Without it a stalled loop leaves someone in a half-open portal with audio playing, no interface and no way back.
The world appeared upside down	WebGL's first texture row is the bottom; the DOM's is the top. <code>UNPACK_FLIP_Y_WEBGL</code> is set at setup <i>and re-asserted on every frame</i> , because it is context state that anything else touching the GL context could reset.
Hover teleported people instantly	There is no dwell timer. Hover peeks; only a click commits. Being teleported by a mouse path that happened to pass the corner is hostile.

The audio ramp

Your brief asked for the world's audio to rise smoothly to a maximum of 40%. That is `volumeCeiling = 0.4` , ramped over 900ms on hover and back down over 420ms.

Two details matter. The ramp runs on `setInterval` , not rAF, for the reason in 5.5. And `sound()` checks a `gestured` flag before making any noise — browsers block audio until the visitor has interacted with the page, so until then a hover opens the portal in silence rather than throwing an error.

THE BUG THAT HID HERE

The gesture flag originally listened for `pointerdown` , `pointerup` , `keydown` and `touchstart` — but not `click` . On some paths the volume snapped to full instead of ramping. If you ever add audio elsewhere, list every event that could count as an interaction.

The CSS bug that made the entire site unclickable

Worth its own heading because it is the most instructive failure in the project.

The world section is marked `hidden` in HTML. The `hidden` attribute works by the browser's own default stylesheet applying `display: none` — and **any rule you write outranks the browser's defaults**. A rule saying `.world { display: flex }` therefore defeated `hidden` entirely. The world stayed laid out, invisible but full-screen, on top of the page, swallowing every click and scroll.

The fix is now a global rule near the top of the stylesheet:

```
[hidden] { display: none !important; }
```

DO NOT DELETE THAT LINE

It looks like a stray one-liner. It is load-bearing for the entire site's interactivity, and it protects every future component that uses `hidden` — the roster view, the join success panel, the crew wrapper.

6.2 The chatbot, honestly described

IT IS NOT AI

It is a **keyword matcher**. It scores what someone typed against the keywords attached to each answer you wrote, and returns the best one. It never generates text. Every word it says is a word you wrote.

This was a deliberate engineering choice, and the reasoning applies to your future projects. A real language model needs an API key. A key shipped inside a public web page can be read by anyone viewing source and used to run up your bill — and there is no way to hide it without a backend. The matcher costs nothing, works offline, never invents an answer, and cannot be abused.

How the matching works

1. **Normalise** — lowercase, strip punctuation, split into words.
2. **Drop filler** — about 80 words carrying no meaning. Question words (*what, how, when*) are in the filler list deliberately: when they counted as keywords, "what is the weather today" matched "What is EDGE?" purely on the word "what".
3. **Stem** — a crude suffix-stripper so "years" matches "year" and "casting" matches "cast". Applied to both sides so they meet in the middle.
4. **Score** — +2 per word matching a keyword, +1 per word appearing in the question text, +3 if the typed text is nearly the whole question.
5. **Normalise by length** — divide by the *square root* of the word count.
6. **Compare to the confidence bar** (1.4). Above it, answer. Below it, say so honestly and offer the nearest questions instead of guessing.

WHY SQUARE ROOT, AND WHY THIS IS A GOOD LESSON

Dividing by the full word count punished natural phrasing far too hard: "i want to sign up" scored below a bare "join", because three of its words carried no signal. The square root still stops a long rambling question from beating a short precise one, but much more gently.

The general lesson: when a scoring system misbehaves, the fix is usually to change *how* you normalise, not to add special cases.

Changing the bot

You almost never edit `faq-bot.js`. To change what it knows, edit `EDGE.faq.entries` in `content.js`. Each entry:

```
{
  q: "How do I join?",
  a: "Fill in the Join form. Pick Member for events and socials, ...",
  keywords: ["join", "sign", "signup", "register", "apply", "become", "entry"]
}
```

Put in the words people *actually type*, including slang and likely misspellings. The existing entries already include "enrol" and "enroll", "csgo" and "cs2", "pubg" for BGMI. That is the level of care that makes the difference.

Every entry also renders as a plain expandable list underneath the chat, so nobody is ever stuck if the bot misunderstands — and so search engines can read your answers.

IF YOU LATER WANT A REAL AI CHATBOT

You cannot do it safely from a static site. You would need a small backend that holds the key and forwards requests. That is a genuine project, not a config change. The current design is the right one until the club has a reason to outgrow it.

6.3 The join form — and why it is broken

What works

Two paths in one form. Member asks the basics; Roster adds the competitive questions. Which fields are required changes with the path, so validation is driven by a rule table (`join.js:93`) rather than by `required` attributes in the HTML — a hidden field with `required` would block submission with a message nobody can see.

The validation is careful: names allow any script plus spaces and apostrophes (`\p{L}` matches letters in every language, so it does not reject non-Latin names); phone must be a 10-digit Indian mobile starting 6–9; email must have a plausible shape. There is also a **honeypot** — an invisible field named `company` at `index.html:174` . A human never sees it, so any value means a bot. The bot is shown the normal success screen rather than being told it was caught.

What is broken

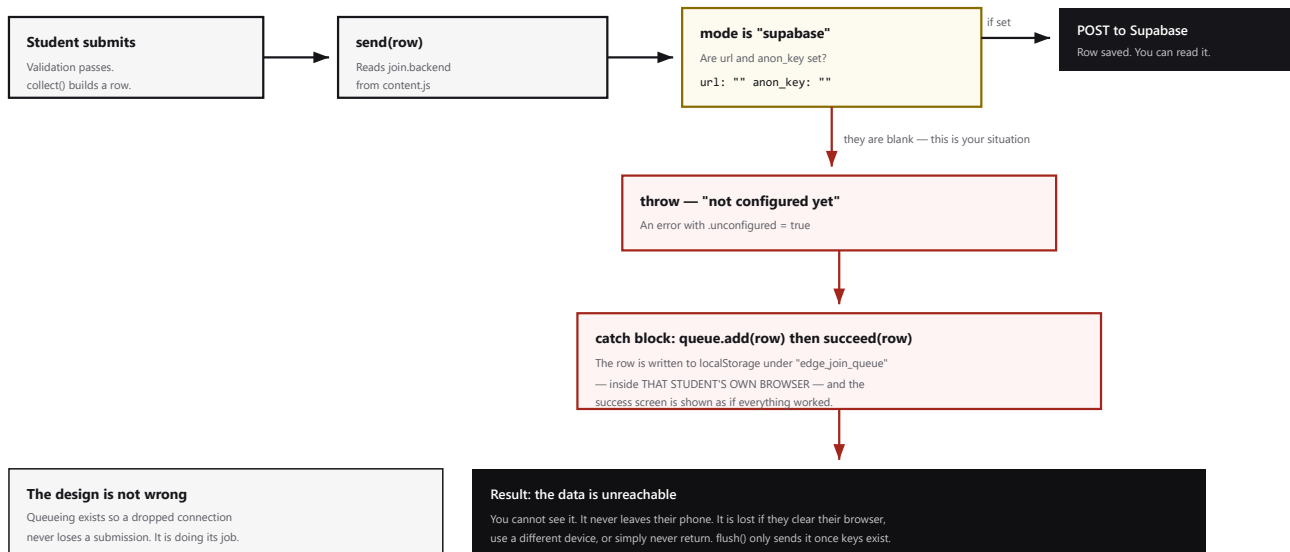


Figure 6. Why a working form produces nothing. The queue is a feature being used as a failure mode.

UPDATED — THE KEYS ARE NOW IN

Project `https://wrjhopokogleplbbxntl.supabase.co` is connected in `content.js`. The diagram above still describes what happens until `supabase-schema.sql` is run: with no table to receive them, the POST returns 404, the catch block queues the row, and the success screen is shown. The path is identical; only the reason changed.

Run the schema, then `bash scripts/verify-supabase.sh`. Both branches of the diagram resolve at that point.

One signal does exist: `join.js:359` prints a warning to the browser console on every page load:

```
[EDGE] The join form is not connected yet.  
Paste your Supabase URL and anon key into js/content.js -> join.backend.supabase.
```

Nobody was looking at the console, which is why this went unnoticed. Part 12 teaches you to look.

The security model — the most important thing in your project

Once you connect Supabase, your anon key sits in `content.js` where anyone can read it. **That is intended.** What protects the data is not secrecy but a database rule.

ROW LEVEL SECURITY (RLS)

A Postgres feature letting the database decide, per row and per operation, who may do what. Supabase publishes every table over a public REST API, so RLS is the only thing standing between your anon key and your data.

`supabase-schema.sql` creates exactly one policy:

```
create policy "public can register"
  on public.registrations
  for insert          -- INSERT only
  to anon             -- for anonymous visitors
  with check (true);
```

And deliberately creates no others. No select policy means **reads are denied by default**. No update policy means nobody can alter a submission. No delete policy means nobody can wipe your registrations. You read the data while signed in to the Supabase dashboard, which runs as the service role and bypasses RLS.

GET THIS WRONG AND YOU LEAK EVERY STUDENT'S PHONE NUMBER

Without that policy — or with a select policy added carelessly — anyone could take the key from your page and download every name, register number, phone number and email address you have collected.

The schema file ends with two `curl` commands to verify it. The insert must succeed; the select **must come back as an empty list** []. Run both before the site handles a single real registration. If the select returns rows, stop and fix it.

The database also re-validates everything the browser checked — phone shape, email shape, and that a roster application has a game and an in-game name. This is not redundancy. **Client-side validation is a convenience for the person filling the form; it is not a guarantee**, because anyone can POST directly at the API and skip your JavaScript entirely. The database constraints are the actual protection.

Hosting, deployment and the outside world

How your files get from your laptop to a student's phone, and every outside party involved on the way.

7.1 Git, GitHub and GitHub Pages — three different things

People use these names interchangeably. They are not the same, and understanding the difference makes deployment obvious.

GIT

A program on *your computer* that records snapshots of your project. Each snapshot is a **commit**, with a message saying what changed and why. Git works entirely offline. It is your undo history, at project scale, forever.

GITHUB

A website that stores a copy of your git repository in the cloud. It is your backup, your collaboration point, and — for you — your deployment mechanism. Your account is `potatoesareathing`; your repository is `edge-website`.

GITHUB PAGES

A feature of GitHub that takes whatever is committed on a chosen branch and serves it as a website. Yours is configured to serve the `main` branch from the **root** folder. It runs no code — it only hands out files.

7.2 What a deployment actually is

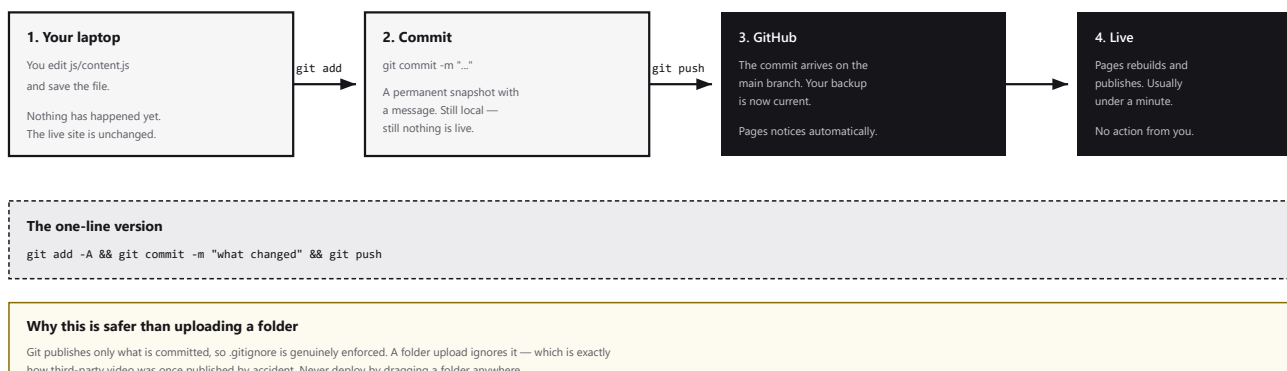


Figure 7. Deployment. Three commands, no dashboard, no build.

The commands you will actually use

Command	What it does
---------	--------------

<code>git status</code>	What have I changed? Run this constantly. It is free and it prevents mistakes.
<code>git diff</code>	Show me the exact lines I changed. Run before every commit.
<code>git add -A</code>	Stage everything for the next commit.
<code>git commit -m "message"</code>	Take the snapshot.
<code>git push</code>	Send it to GitHub. This is the deployment.
<code>git log --oneline</code>	History, newest first. Yours has 15 commits.
<code>git checkout -- path/to/file</code>	Throw away uncommitted changes to one file. Your undo.
<code>git revert <hash></code>	Undo a commit that is already pushed, by making a new commit that reverses it. Safe.

TWO COMMANDS TO AVOID UNTIL YOU UNDERSTAND THEM

`git reset --hard` permanently discards uncommitted work with no undo. `git push --force` can erase history on GitHub, including other people's. Neither is needed for anything in this manual.

7.3 Every external service, and what happens if it fails

Service	Account	Cost	What it does	If it disappears
GitHub + Pages	<code>potatoesareathing</code>	Free	Stores your code; serves the live site; issues the HTTPS certificate.	Site goes down. But you hold a complete copy in your local git repository — every file and every commit. Push it to GitLab, Codeberg or Cloudflare Pages and you are live again the same day. This is your most important account.
Google Fonts	None	Free	Serves Chakra Petch and Archivo on every page load.	Text falls back to a generic sans-serif. The site works; it looks wrong. You can self-host the fonts — Procedure 11.15 — which also removes a third party from every visit.
Instagram	None	Free	Renders pasted post URLs. No API key, no developer account. Script loaded only if a blog entry has a URL — none do today, so it is never contacted.	Embed cards do not render. Nothing else is affected.
Supabase	<code>not created</code>	Free tier	<i>Would</i> store registrations in a Postgres database.	Not connected, so nothing to lose yet. Once live: registrations fail and queue in visitors' browsers. Data is exportable as CSV — export regularly (11.6).

Netlify	aaliyahaijaz2006	Blocked	Former host. No longer serving anything. Deploys are refused: "Account credit usage exceeded".	Nothing. Already replaced. <code>netlify.toml</code> remains in the repo but is inert.
---------	------------------	---------	---	--

READ THAT TABLE AS A DEPENDENCY RISK REGISTER

Only one row can take your site down: GitHub. Everything else degrades. That is an unusually good position — most sites have five or six single points of failure — and it is a direct consequence of having no build step and no dependencies.

7.4 HTTPS, DNS and caching on your current host

HTTPS

Handled entirely by GitHub. Your repository reports `https_enforced: true`, so visitors arriving over plain HTTP are redirected. The certificate is issued and renewed automatically. **Nothing to maintain** — unless you add a custom domain, in which case you must wait for the certificate to be issued and re-tick "Enforce HTTPS".

DNS

You have none. GitHub owns `github.io`. The subpath `/edge-website/` comes from the repository name.

WHY THE SUBPATH WORKS AT ALL

Because every asset path in your site is *relative* — `css/styles.css`, not `/css/styles.css`. A leading slash means "from the domain root", which would resolve to `github.io/css/styles.css` and 404.

Rule: never start an asset path with a slash in this project. It will work in local preview and break in production, which is the worst kind of bug.

Caching

GitHub Pages sets its own cache headers and **you cannot configure them**. The rules in `netlify.toml` do nothing. In practice Pages serves assets with a ten-minute cache, which is short enough that stale files are rarely a problem — but when you do replace an image and still see the old one, add or bump a version marker:

```
 <!-- was ?v=2 -->
```

The browser treats a different query string as a different file. This is why every logo reference in your HTML already carries `?v=2`.

One Pages-specific requirement

GitHub Pages runs files through Jekyll, a site generator, which **ignores any file or folder whose name starts with an underscore**. Your project has none, so it works. If you ever add `_drafts/` or `_assets/`, those files will silently not publish. The fix is an empty file named `.nojekyll` at the repository root.

What happens if Claude disappears tomorrow

The honest answer: your website keeps running, unchanged, indefinitely. Nothing in it calls an AI service. But your ability to *change* it depends on things you should secure now.

8.1 What is genuinely at risk

Concern	Real risk	Why
The site stops working	None	No AI service is contacted at runtime. The chatbot is a keyword matcher running on the visitor's device. Delete every AI tool from your life and the site is unaffected.
You cannot deploy	None	Deployment is <code>git push</code> . That is a standard tool that predates all of this and will outlive it.
You cannot make content changes	Low	<code>content.js</code> is plain data with comments. Part 11 gives you the procedures.
You cannot fix a visual bug	Medium	CSS is learnable and the browser's inspector teaches it to you interactively. Part 12 covers the method.
You cannot modify <code>portal.js</code>	High	Shader programming is a genuine specialism. Realistic mitigation: do not modify it . It works, it is documented, and it degrades gracefully. If it ever breaks beyond repair, delete the script tag — the FAQ world still opens, just without the effect.
You lose the video sources	High	The two <code>-source.mp4</code> files are gitignored and exist only on your laptop. A disk failure loses them permanently. Back them up today.

8.2 What to secure this week

Accounts and credentials

- **GitHub** (`potatoesareathing`) — the critical one. Turn on two-factor authentication. Write down the recovery codes and keep them somewhere physical. If the club will outlive you, add a second committee member as a collaborator now.
- **Netlify** (`aaliyahaijaz2006`) — no longer needed. Keep the login noted in case you return to it.
- **Supabase** — when you create it, record which email owns it. A project tied to a graduating student's personal account is a club asset that walks out the door.
- **Domain registrar** — if you ever buy a domain, this becomes as critical as GitHub, and it has a renewal date that will lapse silently.

Files that exist in only one place

- `assets/video/homeworld-source.mp4` and `faq-world-source.mp4` — the originals before the crossfade edit.
- `_archive-v1/` — the first version of the site.
- The original logo artwork the SVGs were traced from.

Copy all of these to cloud storage or an external drive. They are not in git and GitHub does not have them.

Knowledge that lives only in the documentation

Four files in your repository carry reasoning that is not recoverable from the code: `README.md` (how to set up and deploy), `ARCHITECTURE.md` (thirteen invariants — rules that must stay true), `DESIGN.md` (why it looks this way, and a record of past mistakes), `PRODUCT.md` (who the site is for). Plus this manual.

Keep them updated. Documentation that lies is worse than none, because you will trust it.

8.3 Vendor lock-in, honestly assessed

Dependency	Lock-in	Escape route
AI tooling	None	Nothing in the codebase references it. The code is plain HTML, CSS and JavaScript that any developer — or any future tool — can read.
GitHub Pages	Very low	Your site is static files. Cloudflare Pages, GitLab Pages, Codeberg Pages, Vercel and a \$3/month shared host all serve it unchanged. Migration is: push the same repo elsewhere, point it at <code>main</code> . Under an hour.
Git	Low, and worth it	Git is the industry standard and fully decentralised — your local clone is a complete repository. This is lock-in to an open format, which is the only acceptable kind.
Supabase	Medium, once used	The database is standard PostgreSQL, so the data and the schema are portable. What is Supabase-specific is the REST API shape in <code>join.js:211</code> and the RLS policy syntax. Moving would mean rewriting one function. Export your data as CSV regularly regardless.
Google Fonts	Low	Both fonts are open source. Download the files, put them in <code>assets/fonts/</code> , add an <code>@font-face</code> block, delete the two <code><link></code> tags. Procedure 11.15.
WebGL / the portal	Conceptual	Not vendor lock-in — knowledge lock-in. Nobody can take WebGL away, but you may not be able to change the shader. Treat <code>portal.js</code> as a finished component.

8.4 The recovery drill

Do this once, now, while nothing is wrong. It takes fifteen minutes and it converts "I hope I could recover" into "I have recovered before".

```
cd ~/Desktop
git clone https://github.com/potatoesareathing/edge-website.git edge-test
cd edge-test
node serve.js
# open http://localhost:8790 - the whole site should work
```

If that works, you have proved that GitHub alone is enough to rebuild your entire project on any computer. Delete the test folder afterwards.

THE SUMMARY

Your genuine dependencies are **one GitHub account, two video source files that are not backed up, and your own understanding.** The first needs two-factor authentication and a second owner. The second needs a copy made today. The third is what this manual is for.

The maintenance inventory

Everything you own, in one place. For each item: what it is, why it exists, how you know it needs attention, how to update it, what breaks if you get it wrong, and how often to check. Items are ordered by how likely you are to touch them.

9.1 Content

Item	What & why	How you know	How to update	Risk if wrong	Cadence
Leaderboard	<code>content.js:227</code> . The standings table. Currently all placeholders (<code>PLAYER-01 / Full Name</code>).	A tournament finished; the table is stale or fake.	Edit the array, push. Procedure 11.6.	A missing comma empties the whole site. Wrong <code>move</code> values mislead but break nothing.	After every event
Rosters	<code>content.js:236</code> . Official teams. Also placeholders.	Tryouts concluded; someone left.	Edit <code>players</code> array. Set <code>status:</code> <code>"recruiting"</code> with empty players for a team you are forming.	Same syntax risk. A player named in public who has left is a real-world problem.	Each semester
Updates & events	<code>content.js:307</code> . Drives the list, the archive split, and the countdown.	An event passed, or a new one is announced.	Add an entry with <code>kind</code> , <code>title</code> , <code>date</code> , <code>body</code> . Past events archive themselves.	A wrong date puts an event in the archive immediately, or points the countdown at the wrong thing.	Weekly in season
Blog / from the club	<code>content.js:38</code> . Photos and write-ups. Two entry types.	You ran something and have photos.	Add an entry; put images in <code>assets/blog/</code> . Procedure 11.5.	A wrong <code>media</code> path shows a broken image. Huge unoptimised photos slow the page badly.	After events
About copy	<code>content.js:270</code> . Lead, body, facts, pillars, crew.	The club's description drifts from reality; a new committee.	Edit the strings. Leave <code>crew</code> empty and the section hides itself.	Cosmetic only.	Yearly
FAQ knowledge	<code>content.js:155</code> . The chatbot's entire brain and the static FAQ list.	Someone asks a question the bot fumbles.	Add keywords to an existing entry, or add a new entry. Procedure 11.7.	Too many keywords on one entry makes it swallow unrelated questions.	Monthly

Social links	<code>content.js:15</code> . All three are empty strings today , so the footer shows no links at all.	Look at the footer — it is bare.	Paste full URLs including <code>https://</code> .	A Discord invite that expires becomes a dead link nobody notices.	Quarterly
Branch list	<code>content.js:87</code> . The form's dropdown. Marked <i>TODO confirm</i> in the code.	A student cannot find their branch.	Replace with the exact RUAS names.	Wrong names produce unusable registration data.	Once, then yearly

9.2 Assets

Item	What & why	How you know	How to update	Risk if wrong	Cadence
Logo SVGs	Vector traces of the real artwork. Mark is 398×333 , wordmark 665×183 . Neither is square.	The logo looks squashed, stretched or the wrong size.	Replace the file, bump <code>?v=2</code> to <code>?v=3</code> everywhere in <code>index.html</code> .	High . Wrong aspect ratio distorts it everywhere. Never add a blanket <code>img[src*="edge-mark"]</code> CSS rule — it outranks every component and breaks all sizes.	Rarely
Videos	Two 3.9 MB loops. 7 seconds each, crossfaded so the loop point is invisible. 97% of your page weight .	You want a new look, or the loop visibly jumps.	Procedure 11.2. Must be re-crossfaded, not just trimmed.	High . An uncrossfaded loop jumps every 7 seconds. An oversized file punishes students on mobile data.	Rarely
Video posters	Still frames shown before the video loads, and permanently for anyone on a saving connection.	You changed a video and the poster no longer matches.	Extract a frame with ffmpeg, save as jpg.	Low, but a mismatched poster is a visible flash.	With the video
Video sources	The uncut originals. Gitignored — not in GitHub, not backed up anywhere .	You need to re-edit and cannot find them.	Nothing to update — just copy them to cloud storage today .	Permanent loss on disk failure.	Back up now
Blog images	<code>assets/blog/</code> — referenced by <code>content.js</code> . The folder does not exist yet.	A card shows a broken image.	Resize to about 1600px wide, compress, then add. Procedure 11.5.	Unoptimised phone photos are 4–8 MB each and will make the page unusable on mobile.	With posts
Favicon	Does not exist . The browser tab shows a blank page icon.	Look at your tab.	Procedure 11.18.	None.	Once

9.3 Code and configuration

Item	What & why	How you know	How to update	Risk if wrong	Cadence
Dependencies	You have none. No <code>package.json</code> , no <code>node_modules</code> .	Not applicable.	Nothing to do. This is the single biggest maintenance saving in the project.	The risk is <i>adding</i> one. Every dependency is a thing that can break, get abandoned, or ship a security flaw.	Never
Framework versions	No framework. Plain browser JavaScript.	Not applicable.	Nothing.	None.	Never
Browser compatibility	Your code uses <code>replaceChildren</code> (2020), optional chaining <code>?.</code> (2020), <code>IntersectionObserver</code> , WebGL, Web Audio.	Someone reports a blank or broken section on an old phone.	Test on a real old Android occasionally. Every advanced feature already has a fallback.	Low. A browser older than about 2020 would struggle, but the graceful degradation means it fails softly.	Yearly
Security patches	Nothing to patch in your own code — no server, no dependencies, no login.	Not applicable to the frontend.	Your only real security surface is the Supabase RLS policy.	See 9.4.	—
<code>.gitignore</code>	Keeps third-party video and the untracked sources out of the public repo.	You add files that should not be public.	Add a line before the first commit of that file.	High. Git history is permanent — publishing something once cannot be fully undone.	When adding files
<code>netlify.toml</code>	Inert. Old host's configuration. GitHub Pages ignores it.	You believe its cache rules are in effect. They are not.	Keep or delete. Do not rely on it.	Only confusion.	—
Documentation	README, ARCHITECTURE, DESIGN, PRODUCT, and this manual.	Something in them contradicts the code.	Edit as part of the same commit as the change.	Underrated. Documentation that lies is worse than none.	With every change

9.4 Data, services and security

Item	What & why	How you know	How to update	Risk if wrong	Cadence
Supabase connection	<code>content.js:126</code> . Project <code>wrjhopokogleplbbxntl</code> , connected. The remaining step is running the schema so the table exists.	<code>bash</code> <code>scripts/verify-supabase.sh</code> — it tells you in one second.	Procedure 11.14, steps 2 and 5 onward.	Registrations still queue in the visitor's browser until the table exists.	Finish now, then quarterly

The anon key	Public by design. Visible in your source. Safe <i>only</i> because of RLS.	You rotate it, or suspect abuse.	Rotate in the dashboard, paste the new value, push. Procedure 11.13.	Low on its own — unless RLS is misconfigured, in which case it is the key to all your data.	If compromised
The service role key	Bypasses all security. Must never appear in this project in any file.	If it is ever in your repo, treat it as compromised immediately.	Rotate it in Supabase at once.	Catastrophic. Full read, write and delete on all data.	Never commit
RLS policy	Insert-only for anonymous visitors. The only thing protecting student data.	Run the two <code>curl</code> checks in <code>supabase-schema.sql</code> . The select must return <code>[]</code> .	SQL Editor in the dashboard.	Catastrophic. A careless select policy exposes every phone number and email.	Verify at launch, then quarterly
Registration data	Real personal data about real students: names, register numbers, phones, emails.	You need the list; or a student asks to be removed.	Read and export CSV from the Supabase dashboard.	Legal and ethical. Do not share the export. Delete it when finished.	Export monthly
Backups	Code is backed up by GitHub automatically. Registrations are not. Video sources are not.	Nothing tells you. That is the danger.	Procedure 11.11.	High. Free-tier databases can be paused or deleted for inactivity.	Monthly
HTTPS certificate	Issued and renewed by GitHub automatically.	A browser warning. Extremely unlikely.	Nothing, unless you add a custom domain.	None today.	—
Hosting quota	GitHub Pages soft limits: 1 GB site, 100 GB bandwidth/month. You use 8 MB.	GitHub emails you. You are at under 1% of both.	Nothing.	None at your scale. Note that videos dominate bandwidth: 1,000 visits who all watch is about 8 GB.	Yearly

9.5 Quality: the things nobody asks about until they break

Item	Current state	How you know	How to update	Risk if wrong	Cadence
Performance	~8 MB total, but only ~200 KB before the video is conditionally attached. Good.	The site feels slow on a phone on campus wifi.	Compress images before adding; never remove the connection checks around the video.	Medium. Adding big unoptimised images is the realistic way this degrades.	After adding media

Accessibility	Genuinely good: ARIA tabs with arrow keys, <code>alt</code> text, labelled fields, <code>role="alert"</code> errors, full reduced-motion support.	Navigate the whole site using only Tab and Enter.	Keep new controls to real <code><button></code> elements. Never use a <code><div></code> as a button.	Medium. Easy to erode one careless addition at a time.	After UI changes
SEO	Title and description set. No <code>robots.txt</code> , no <code>sitemap.xml</code> , and one page means one search result.	Search for "EDGE RUAS esports" and see what appears.	Improve the description in <code>index.html</code> once content is real.	Low.	Yearly
Analytics	None. You have no idea how many people use the site.	You cannot answer "did the club fair drive traffic?"	Optional. A privacy-respecting counter like GoatCounter is one script tag.	Adding a tracker has privacy implications; students deserve consideration.	Optional
Error monitoring	None. If the site breaks for a visitor, nothing tells you.	Only from a complaint.	Optional. The realistic substitute: open the site on a real phone monthly and watch the console.	Medium. You could be broken for weeks without knowing.	Monthly manual check
Logs	No server logs exist — you have no server. The only logs are the browser console on the visitor's own device.	Not applicable.	Nothing.	None, but understand the implication: you cannot investigate a problem you cannot reproduce.	—
Forms & email	One form. No email system at all — nothing notifies you of a registration, and nothing confirms to the student.	You have to check the dashboard manually.	Would require a backend or a Supabase Edge Function. A real project.	Medium: a registration nobody notices is a student who thinks they were ignored.	Consider later
Domain & DNS	None. Using GitHub's subdomain.	Not applicable today.	Procedure 11.17 if you buy one.	Once you own a domain, a missed renewal takes the site down and the name can be bought by anyone.	Annual renewal

Known gaps and unfinished work

Everything the code itself flags as incomplete, plus everything that could not be determined from the project files. Nothing here is guessed at.

10.1 Marked TODO in the code

Where	What is missing	Consequence today
<code>content.js:126</code>	Supabase url and anon_key are empty strings. Done — project <code>wrjhopokogleplbbxntl</code> is connected.	The remaining step is running <code>supabase-schema.sql</code> so the table exists, then <code>scripts/verify-supabase.sh</code> . Until the schema is run, submissions still queue in the visitor's browser.
<code>content.js:227</code>	Leaderboard is five rows of <code>PLAYER-01 / Full Name</code> with zero points.	The Players tab shows obvious placeholder data to anyone who visits.
<code>content.js:236</code>	Both rosters are placeholder players.	Same.
<code>content.js:302</code>	<code>crew</code> is an empty array.	The "Who runs it" section hides itself. Intentional behaviour, but the club has no named committee on its own site.
<code>content.js:15</code>	Instagram, Discord and YouTube links are empty.	The footer renders no social links at all.
<code>content.js:20</code>	Club email <code>edge@msruas.ac.in</code> marked <i>TODO confirm</i> .	May not be a real address. If it bounces, the site's only contact route is dead.
<code>content.js:87</code>	Branch list marked <i>TODO replace with the exact names RUAS uses</i> .	Students may not find their branch; data quality suffers.
<code>content.js:289</code>	Founded year "2026" marked <i>TODO confirm</i> .	A factual claim on the About page that may be wrong.
<code>content.js:38, 307</code>	Blog posts and updates are written as plausible examples, marked <i>TODO</i> .	The site describes events that may not have happened.
<code>join.js:103</code>	Register number validation deliberately permissive, <i>TODO tighten once the RUAS format is confirmed</i> .	Accepts almost anything 4–20 characters. Low risk, but typos pass.

THE CONTENT PROBLEM, STATED PLAINLY

Your site is architecturally finished and factually empty. Every system works; almost every piece of information in it is a placeholder. Before you show this to students, one afternoon with `content.js` matters more than any code change in this manual.

10.2 Needs verification

These could not be determined from the project files. Do not assume either way.

Question	Why it is unresolved
Does a Supabase project exist at all? Resolved	It does: <code>https://wrjhopokogleplbbxntl.supabase.co</code> , connected in <code>content.js</code> and verified reachable. The anon key decodes to <code>role: anon</code> and its project ref matches, so it is the right key for the right project. Still unresolved: whether <code>supabase-schema.sql</code> has been run. At the time of writing the table did not exist — the API answered <code>Could not find the table 'public.registrations'</code> . Run <code>scripts/verify-supabase.sh</code> ; it answers this in one second.
Is <code>edge@msruas.ac.in</code> a working address?	Marked TODO in the code. Send it a test email.
Was the club actually founded in 2026?	Marked TODO. Confirm with the committee.
What ffmpeg command produced the crossfaded loops?	Not recorded anywhere in the project. The technique is documented in <code>DESIGN.md</code> ; the exact invocation is not. Procedure 11.2 gives a command that should reproduce it — test before relying on it.
Who else has access to the GitHub account?	Not determinable from the repository. Check the account's settings.
Does the club own any domain name?	No <code>CNAME</code> file exists and no domain appears anywhere in the project, which strongly suggests no — but a domain could be registered and simply unused.
Is there a privacy notice or consent requirement?	The form collects personal data from students. The site says only "Your details are used only to contact you about EDGE." Whether RUAS requires more is an institutional question, not a technical one. Ask before collecting at scale.

10.3 Things that work but are worth knowing about

- **The back button skips tab changes.** `history.replaceState` replaces rather than adds history entries. Deliberate, but it surprises people.
- **The leaderboard hides two columns on phones.** W and L are dropped below 680px because they cost 150px of width and are derivable from the record. The full table is there on a laptop.
- **A comment in `portal.js` says "two triangles"** but the code uploads three vertices and draws one oversized triangle covering clip space — a standard trick that avoids a seam down the diagonal. The comment is slightly wrong; the code is correct.

- `serve.js` **is published but never runs.** GitHub Pages executes nothing. Harmless, but do not expect it to do anything in production.
- `EDGE_ASK` **and** `EDGE_SOUND` **are exposed but unused.** Deliberate hooks for features not yet built.

The maintenance playbook

Eighteen procedures for the jobs you will actually do, written against your real files. Every one ends the same way, so learn that ending once.

THE ENDING, USED BY EVERY PROCEDURE BELOW

```
node serve.js                # preview at http://localhost:8790
# check it, then:
git status                   # what changed?
git diff                     # exactly which lines?
git add -A
git commit -m "describe what changed"
git push                     # this is the deployment
# wait ~60s, then hard-refresh the live site: Ctrl+Shift+R
```

Referred to below as **[preview, commit, push]**.

11.1 Change homepage text

1. The hero line ("Esports and gaming at Ramaiah...") is written directly in `index.html`, in `<p class="home__line">`. Edit it there.
2. Everything *below* the hero comes from `content.js` → `about : lead`, `body` (an array of paragraphs), `facts`, `pillars`.
3. [preview, commit, push]

What breaks: nothing structural. If the About section goes blank, you broke JSON syntax in `content.js` — see 12.6.

11.2 Replace a video

The hard part is not swapping the file; it is the crossfade that makes the loop invisible.

1. Keep your new source out of the repo: name it `homeworld-source.mp4`, which `.gitignore` already excludes.
2. Produce a crossfaded loop. For an 8-second source producing a 7-second loop:

```
ffmpeg -i homeworld-source.mp4 -filter_complex ^
"[0:v]trim=0:1,setpts=PTS-STARTPTS[head]; ^
[0:v]trim=7:8,setpts=PTS-STARTPTS[tail]; ^
[tail][head]blend=all_expr='A*(1-T/1)+B*(T/1)'[fade]; ^
[0:v]trim=1:7,setpts=PTS-STARTPTS[mid]; ^
[fade][mid]concat=n=2:v=1[out]" ^
-map "[out]" -an -c:v libx264 -crf 26 -pix_fmt yuv420p homeworld.mp4
```

Needs verification — this reconstructs the technique; test the output before shipping.

3. Check the size. Target under 4 MB. Raise `-crf` (28, 30) to shrink it.
4. Extract a matching poster:

```
ffmpeg -i homeworld.mp4 -ss 2 -vframes 1 -q:v 4 homeworld-poster.jpg
```

5. Put both in `assets/video/`, replacing the old ones. Filenames must stay the same, or update `VIDEO_SRC` at `app.js:735`.
6. **Test the loop:** watch it for 30 seconds. If you can see where it restarts, the crossfade failed.
7. [preview, commit, push]

What breaks: the FAQ world's video *must keep its audio track* — the portal ramps its volume to 40%. Adding `-an` to that one would silence the feature.

11.3 Add a page

Your site has no pages, only tabs. Adding a genuine second HTML file would break the single-page model.

Add a tab instead — 11.4.

If you truly need a separate URL (for sharing, or for search engines), create `rules.html` as a copy of `index.html` with the body replaced, keeping the same stylesheet and script tags. Be aware you now maintain the header and footer in two places, which is how sites drift out of sync.

11.4 Add a navigation tab

Four edits, in order. Miss one and the tab appears but does nothing.

1. `index.html` — add the button next to the others, copying every attribute:

```
<button class="tab" role="tab" id="tab-rules" aria-controls="panel-rules"
        aria-selected="false" data-tab="rules">Rules</button>
```

2. `index.html` — add the panel inside `<main>`:

```
<section class="panel" id="panel-rules" role="tabpanel"
        aria-labelledby="tab-rules" tabindex="0">
  <div class="panel__head"><h1 class="painted">Rules</h1></div>
  <div id="rulesBody"></div>
</section>
```

3. `app.js:45` — add the name to the router:

```
const TABS = ["about", "players", "updates", "join", "rules"];
```

4. **Content** — either write it into the HTML directly, or add a `rules` key in `content.js` and a small renderer in `app.js` called from `init()`.
5. [preview, commit, push]

What breaks: forget step 3 and clicking the tab does nothing — `showTab` falls back to About because the name is not in `TABS`. Forget the ARIA attributes and the tab is unusable with a screen reader.

11.5 Add a blog post with a photo

1. **Optimise the image first.** A phone photo is 4–8 MB. Resize to about 1600px wide and save as JPEG at quality 80 — usually 200–400 KB. Use Squoosh (squoosh.app), GIMP, or: `ffmpeg -i big.jpg -vf scale=1600:-1 -q:v 5 small.jpg`
2. Create `assets/blog/` if it does not exist. Use lowercase names with hyphens.
3. Add the entry at the top of `EDGE.blog`:

```
{
  type: "post",
  title: "Valorant tryouts - day two",
  date: "2026-10-04",
  tag: "Events",
  media: "assets/blog/tryouts-day2.jpg",
  alt: "Players at the tryouts, seated at the lab machines",
  body: "Twenty-two players across four hours.",
  credit: "Ananya R"
}
```

4. [preview, commit, push]

What breaks: a wrong `media` path shows a broken image. Skipping `alt` makes the post invisible to screen readers. Skipping optimisation is the single most common way a fast site becomes slow.

For an Instagram post instead, use `type: "instagram"` and paste the post URL into `url`. No API key is needed. An empty `url` is skipped silently.

11.6 Update the leaderboard and rosters

1. Open `content.js`, find `players.leaderboard` (line ~227).
2. Replace each row. `move` is the change in position since the last update — positive is a climb, negative a drop, 0 unchanged:

```
{ ign: "ZENITH", name: "Rahul Menon", game: "Valorant",
  points: 1420, w: 18, l: 6, move: 2 }
```

3. Order does not matter for display order — the table renders in array order, so **put them in the order you want them shown**.
4. For rosters, find `players.rosters`. Set `status: "recruiting"` with an empty `players: []` for a team you are still forming; it shows a tryout call instead.
5. [preview, commit, push]

What breaks: a missing comma or unclosed quote empties the *entire site*, because `content.js` fails to parse and every renderer loses its data. Loud and immediately obvious. Fix: 12.6.

11.7 Change what the chatbot knows

Do not edit `faq-bot.js`. Edit `EDGE.faq.entries` in `content.js`.

To improve an answer the bot already gives

Edit the `a` string. Nothing else changes.

To fix a question the bot misunderstands

Find the entry that *should* have matched and add the words the person actually typed to its `keywords`, including slang and misspellings.

To add a whole new question

```
{
  q: "Can I join two teams?",
  a: "No. You can try out for more than one title, but you play for one roster per season.",
  keywords: ["two", "multiple", "both", "second", "another", "team", "teams", "switch"]
}
```

It appears in the chat *and* in the static "Every question" list automatically.

If the bot answers too eagerly or too rarely

Only then touch `faq-bot.js:96: CONFIDENCE_BAR = 1.4`. Raise it to make the bot more cautious, lower it to make it guess more. Change it in steps of 0.1 and test.

What breaks: piling keywords onto one entry makes it swallow unrelated questions. If one answer starts appearing for everything, that entry has too many generic keywords.

11.8 Add an event or deadline

1. Add to `EDGE.updates`:

```
{
  kind: "deadline",           // deadline | event | announcement | result
  pinned: true,               // optional - floats it to the top
  title: "BGMI tryouts - registration closes",
  date: "2026-10-12",         // YYYY-MM-DD, always
  time: "23:59",              // optional, drives the countdown precisely
  venue: "Main Campus",       // optional
  body: "Submit the Join form on the Roster path before this closes."
}
```

2. [preview, commit, push]

The countdown retargets automatically to the nearest future deadline. Past events move to the archive on their own. **You never manually archive anything.**

What breaks: a wrong date silently archives the event immediately, or aims the countdown at the wrong thing. Always use `YYYY-MM-DD`.

11.9 Change an image or the logo

1. **Check the dimensions of what you are replacing.** The mark is 398×333 and the wordmark 665×183. Neither is square.
2. Replace the file, keeping the same name.
3. **Bump the version marker** in `index.html` — every occurrence of `edge-mark.svg?v=2` becomes `?v=3`. There are six references to the mark and four to the wordmark.
4. [preview, commit, push], then hard-refresh.

DO NOT "FIX" LOGO SIZING WITH A BLANKET CSS RULE

Adding `img[src*="edge-mark.svg"] { height: ... }` looks like a tidy global fix. An attribute selector outranks a class, so it overrides every component's own height and breaks the mark everywhere at once. This has already happened once. Each use sets its own height and lets width follow.

11.10 Update dependencies

You have none. There is nothing to update, and that is the correct state. If someone proposes adding a package, ask what it does that 30 lines of your own code could not.

11.11 Back the project up

What	How	How often
All code and assets	<code>git push</code> — GitHub holds a complete copy including every past version.	Automatic, every deployment
Registration data	Supabase dashboard → Table Editor → Export CSV. Store it somewhere private.	Monthly , and before any schema change
Video sources	Copy <code>*-source.mp4</code> to cloud storage. Not in git.	Once — do it today
<code>_archive-v1/</code>	Copy it somewhere. Gitignored.	Once
A full offline snapshot	<code>git bundle create edge-backup.bundle --all</code> — one file containing the entire repository and its history.	Each semester

11.12 Restore from a backup

Undo uncommitted changes to one file

```
git checkout -- js/content.js
```

Undo a commit that is already live

```
git log --oneline          # find the bad commit's hash
git revert a1b2c3d         # creates a NEW commit undoing it
git push
```

Use `revert`, not `reset`. It is safe on published history.

Recover the whole project onto a new computer

```
git clone https://github.com/potatoesareathing/edge-website.git
```

Recover a single deleted file from history

```
git log --oneline -- js/portal.js      # find a commit that had it
git checkout a1b2c3d -- js/portal.js
```

11.13 Change an API key

1. Supabase dashboard → Project Settings → API. Copy the new **anon public** key.
2. Paste into `content.js` → `join.backend.supabase.anon_key`.
3. [preview, commit, push]. Submit one test registration and confirm it appears in the dashboard, then delete the test row.

What breaks: between rotating the key and pushing, every submission fails and queues in visitors' browsers. Do it quickly, and at a quiet time.

Never paste the *service role* key here. It bypasses all security.

11.14 Turn the registration form on — do this first

The most important procedure in this manual. About twenty minutes.

1. **Create the project.** supabase.com → sign up → New project. Choose the region nearest India. Save the database password somewhere safe; you will rarely need it but cannot recover it.
2. **Create the table.** SQL Editor → New query. Paste the entire contents of `supabase-schema.sql` → Run. It creates the table, the constraints, and the security policy.
3. **Copy the two values.** Project Settings → API. You need the **Project URL** (`https://abcdefghi.supabase.co`) and the **anon public** key.
4. **Paste them in** at `content.js:126`:

```
backend: {
  mode: "supabase",
  supabase: {
    url:      "https://abcdefghi.supabase.co",
    anon_key: "eyJhbGciOi...",
    table:    "registrations"
  }
}
```

5. **Verify the security — do not skip this.** Run the two commands at the bottom of `supabase-schema.sql`. The insert must succeed. **The select must return an empty list []**. If it returns rows, RLS is not protecting you: stop, fix it, and do not go live.
6. **Test end to end.** [preview, commit, push], then submit a real registration on the live site. Confirm the row appears in the dashboard. Delete the test row.
7. **Check the console.** The warning *"The join form is not connected yet"* should be gone.

ANYONE WHO REGISTERED BEFORE THIS

Their data is in `localStorage` on their own device and will be sent automatically the next time they open the site — `flush()` runs on every page load. Anyone who never returns is lost. If you ran a club fair before connecting, assume those registrations are gone and collect again.

11.15 Self-host the fonts

Removes a third party from every page load, makes the site work fully offline, and avoids the flash of fallback text.

1. Download Chakra Petch and Archivo from fonts.google.com (both are open source).
2. Put the `.woff2` files in `assets/fonts/`.
3. Add to the top of `styles.css`:

```
@font-face {
  font-family: "Chakra Petch"; font-weight: 700; font-display: swap;
  src: url("../assets/fonts/ChakraPetch-Bold.woff2") format("woff2");
}
```

Repeat for each weight you use: Chakra Petch 500/600/700, Archivo 400/500/600.

4. Delete the three `<link>` tags for Google Fonts from `index.html`.
5. [preview, commit, push]

What breaks: miss a weight and that text silently falls back. Check headings, buttons and body text after.

11.16 Add a second portal world

The portal was built to be reused. To send Players into its own dimension:

1. Produce a crossfaded loop for it (11.2) and add it as `assets/video/players-world.mp4` plus a poster.
2. In `index.html`, copy the `<video class="portal-src">` element, giving it a new `id`. Copy the `<section class="world">` block and give it a new `id` and its own content.
3. In `app.js`, copy `initAsk()` to `initPlayersWorld()`, changing the four element ids and the video path. Call it from `init()`.
4. [preview, commit, push]

ONE REAL CONSTRAINT

`portal.js` holds a *single* set of module-level state — one `video`, one `origin`, one `progress`. Two worlds can exist, but only one may be open or peeking at a time. Opening a second while the first is peeking will confuse it. If you need genuinely concurrent portals, the module must be refactored to create instances.

11.17 Connect a custom domain

1. Buy the domain (Namecheap, Cloudflare, or an Indian registrar for a `.in`). Budget about ₹800–1,500 per year and **note the renewal date**.
2. At the registrar, add DNS records pointing at GitHub:

Type	Name	Value
A	@	185.199.108.153
A	@	185.199.109.153
A	@	185.199.110.153
A	@	185.199.111.153
CNAME	www	potatoesareathing.github.io

Verify these IPs against GitHub's current documentation — they change rarely but they do change.

3. GitHub repo → Settings → Pages → Custom domain → enter it → Save. This creates a `CNAME` file in your repository.
4. Wait for DNS to propagate (minutes to 24 hours), then tick **Enforce HTTPS**.
5. **Your asset paths already work**, because they are relative. Moving from a subpath to a domain root changes nothing in your code.

What breaks: forgetting to re-tick Enforce HTTPS leaves the site on plain HTTP. A lapsed renewal takes the site down and frees the name for anyone to buy.

11.18 Add a favicon

1. Export the EDGE mark as a 32×32 and a 180×180 PNG on a dark background.
2. Save as `assets/favicon-32.png` and `assets/apple-touch-icon.png`.
3. Add to `<head>` in `index.html`:

```
<link rel="icon" href="assets/favicon-32.png" sizes="32x32">
<link rel="apple-touch-icon" href="assets/apple-touch-icon.png">
```

4. [preview, commit, push]. You may need to close and reopen the tab to see it.

Troubleshooting: how to find a fault yourself

The instinct when something breaks is to describe the symptom to an AI and ask for a fix. That works until it doesn't, and it never teaches you anything. This part teaches the alternative: **locate the fault before you try to fix it.**

12.1 The discipline

Debugging is not guessing repeatedly. It is narrowing a search space. Every step should *eliminate* possibilities, and you should be able to say what you learned from each one.

1. **Reproduce it.** Exactly what did you click, on what device, in what browser? A bug you cannot reproduce is one you cannot verify you fixed.
2. **Locate the layer.** Which of the seven layers in 12.2 is it in? Most of your time should go here.
3. **Read the error.** Error messages are precise. Beginners skim them; the message almost always names the file and the line.
4. **Form one hypothesis** and test it. One at a time, or you will not know which change worked.
5. **Fix, verify, and write down what it was.** A bug that recurs and is re-diagnosed from scratch has cost you twice.

12.2 Which layer is it in?

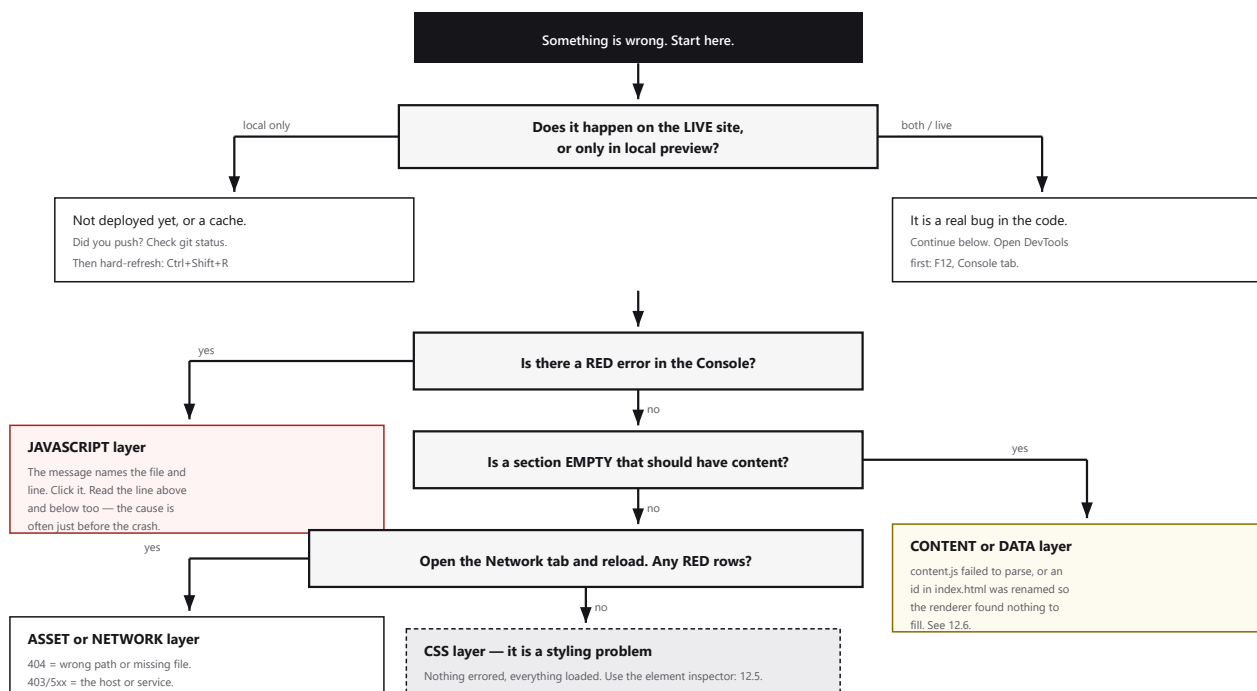


Figure 8. Narrow before you fix. Each question eliminates whole categories.

12.3 Browser DevTools — the four tabs that matter

Press **F12** (or Ctrl+Shift+I). This is the single most valuable tool you own and it is already installed.

Tab	What it shows	Use it for
Console	Errors, warnings, and anything your code logged. Look here first, always.	Red text = JavaScript crashed. It names the file and line. Your site also writes deliberate messages here — the join form's <i>"not connected yet"</i> warning appears on every load right now.
Elements	The live DOM, and every CSS rule applying to whatever you select, in priority order.	Any "why does it look like that" question. Rules that lost on specificity appear struck through . You can edit values live to experiment — changes vanish on refresh, which makes it a safe sandbox.
Network	Every file requested, its status code, size and timing. Reload with it open.	Is the video actually downloading? Did the CSS load? Is the Supabase request being made, and what did it answer? Red rows are failures.
Application	Stored data. Under Local Storage you will find <code>edge_join_queue</code> .	Seeing whether registrations are stuck in the queue on this device — direct evidence of the form problem in 6.3.

ON A PHONE YOU CANNOT PRESS F12

Connect an Android phone by USB, open `chrome://inspect` on your laptop, and you get the full DevTools for the phone's browser. For iPhone, enable Web Inspector in Safari settings and use Safari on a Mac.

Failing that, resize your desktop browser window to phone width. It catches most layout problems, though not touch-specific or performance ones.

12.4 Symptom to cause, for your specific site

Symptom	Most likely layer	Check, in this order
Whole site is blank or unstyled	CSS or HTML	Network tab: did <code>styles.css</code> load (200, not 404)? Console: an HTML syntax error can break parsing. Try the last commit: <code>git stash</code> then reload.
Every dynamic section is empty	<code>content.js</code>	Console will show a syntax error naming a line in <code>content.js</code> — almost always a missing comma, an unclosed quote or a stray bracket. See 12.6.
One section is empty, others fine	HTML id, or that renderer	Did you rename an id in <code>index.html</code> ? The renderer calls <code>byId("blogGrid")</code> , gets <code>null</code> , and returns silently. Compare against the id list in 4.3.
Nothing is clickable anywhere	CSS stacking	An invisible element is covering the page. In Elements, hover nodes to see what highlights over the content. This is the <code>[hidden]</code> bug from 6.1 — confirm <code>[hidden] { display: none !important; }</code> still exists in <code>styles.css</code> .

Portal does nothing on hover	WebGL or the video	Console for a <code>[portal] shader:</code> warning. Network tab: is <code>faq-world.mp4</code> downloading? Test <code>EDGE_PORTAL.supported()</code> in the console — <code>false</code> means no WebGL, and the fallback should still open the world plainly.
Portal opens but is silent	Autoplay policy	Expected if the visitor has not clicked anything yet. Click once, then hover. If it is still silent, check the video actually has an audio track.
Background video never plays	Deliberate, probably	Three gates in 5.4: is it the About tab? Is reduced motion on? Is the connection flagged as data-saving? All three are intentional. Then check Network for the mp4.
Form says success but nothing arrives	Configuration	This is your current state. Console shows the warning. Application tab → Local Storage → <code>edge_join_queue</code> holds the rows. Fix: 11.14.
Form rejects a valid input	Validation rules	<code>join.js:93</code> . Phone must be 10 digits starting 6–9. Register number must be 4–20 characters of letters, digits, slash or hyphen.
Chatbot gives the wrong answer	Keywords	Not a bug — a tuning issue. Add the words the person typed to the right entry's <code>keywords</code> . See 11.7.
Logo looks wrong size or squashed	CSS or cache	Elements tab: check <code>naturalWidth</code> is 398. If it is 200, you are seeing a cached old file — bump <code>?v=</code> . If the natural size is right, a CSS rule is overriding the component height (4.6).
Change pushed but site unchanged	Cache or deployment	Hard-refresh (Ctrl+Shift+R). Check GitHub shows your commit. Check repo → Actions for a failed Pages build. Wait two minutes.
Works on desktop, broken on phone	CSS media queries	Section 20 of <code>styles.css</code> . Remember specificity (4.6) — a phone override that appears later can still lose.

12.5 Diagnosing a CSS problem properly

CSS problems feel mysterious because nothing errors. The method removes the mystery:

1. Right-click the misbehaving element → **Inspect**.
2. The Styles panel lists every rule affecting it, **strongest first**.
3. Find the property that is wrong. If your intended rule appears ~~struck through~~, it lost on specificity — the winning rule is above it. Match its selector shape (4.6).
4. If the property is not listed at all, your selector does not match. Check for a typo or a missing parent class.
5. Edit the value directly in the panel to confirm your fix works, *then* make the same change in `styles.css` .
6. Use the **Computed** tab to see the final value of everything, and the box-model diagram to see actual padding, border and margin.

12.6 The most likely error you will ever cause

A syntax error in `content.js` . It empties the entire site, which is alarming and completely recoverable.

```
Uncaught SyntaxError: Unexpected token '}'    content.js:241
```

Almost always one of three things:

- **A missing comma** between two entries in an array.
- **An unescaped apostrophe** inside a single-quoted string: `'don't'` ends the string early. Use double quotes: `"don't"`.
- **A missing closing bracket** `}` or `]`.

The reported line number is where the parser *noticed*, which is often a line or two after the real mistake. Look just above it.

```
git diff js/content.js      # see exactly what you changed
git checkout -- js/content.js # or: throw it all away and start again
```

12.7 Deciding whether it is your fault

Test	What it tells you
Does it break in a private/incognito window?	No → it is a cache or an extension on your machine, not a real bug.
Does it break in a different browser?	No → a browser-specific issue, usually a feature one browser lacks.
Did it work at the last commit?	<code>git stash</code> , reload, <code>git stash pop</code> . If the old version works, your change caused it — and <code>git diff</code> shows you exactly what you did.
Is the file reaching the browser at all?	Network tab. A 404 is a path problem, not a code problem.
Is GitHub itself down?	<code>githubstatus.com</code> . Rare, but it happens, and there is nothing to fix.

12.8 When you do ask for help

Asking well is a skill, and it is the same skill whether you are asking a person, a forum or an AI. Include:

- **What you expected** and **what happened instead**.
- **The exact error text**, copied, not described.
- **Which layer you have already ruled out**, and how.
- **The relevant code**, not the whole file.
- **What you already tried**.

That list is not politeness. Writing it out solves a surprising share of problems before you finish — the act of stating precisely what you expected is often where the wrong assumption reveals itself.

THE RULE THAT MAKES YOU INDEPENDENT

Never apply a fix you do not understand. If something is suggested and you cannot say why it works, ask for the reasoning before applying it. A site held together by changes you cannot explain is a site you cannot maintain — and you will have simply traded one dependency for another.

Things to understand before your next website

Concepts this project already taught you, made explicit so they transfer. Each one is stated as a working principle, not a definition, with the evidence from your own code.

13.1 Separation of concerns

Your site keeps content, appearance and behaviour in separate files. That is why you can update the leaderboard without touching a line of logic, and why the whole design was converted from colour to monochrome without breaking a single feature.

The test: to change one thing, how many files must you open? If a content change requires editing logic, the separation has failed.

13.2 One source of truth

Every piece of content lives in exactly one place. The leaderboard is in `content.js` and nowhere else. If the same fact lived in two files, they would eventually disagree, and the bug would be invisible until someone noticed the site contradicting itself.

Apply it everywhere: colours as CSS variables, not repeated hex codes. Dates in one format. One list of tab names (`TABS` in `app.js`) driving the router.

13.3 Components are just functions that build things

You do not need React to have components. `postCard(p)` takes data and returns a finished DOM node. Called ten times, you get ten cards. That is the idea; frameworks add lifecycle management and automatic updates on top of it.

The lesson: understand the pattern before you adopt the tool. People who learn React first often cannot explain what it is doing for them.

13.4 State is where bugs live

Your site has very little state, and it is deliberately confined: the current tab, the join path, the portal's progress. Each lives in one place with one owner.

The rule that follows: never store the same truth twice. The portal knows its progress; the CSS reads it. If both JavaScript and CSS each tracked "is the portal open", they would drift apart, and the resulting bug would be maddening.

13.5 Design for failure, because it will

This is the most valuable thing in your codebase. Count the fallbacks:

- No WebGL → the world still opens, plainly.
- Video will not load → the poster image stands in.
- Reduced motion → every effect is off and nothing is downloaded.
- Scroll observer never fires → a two-second timer shows everything anyway.
- Portal stalls → a 2.6-second timer lands it rather than freezing part-open.
- Network drops mid-submission → the row is queued and retried.
- Instagram post URL missing → the card is skipped silently.
- Google Fonts unreachable → fallback fonts.

The principle: for every feature, ask what happens when it fails, and make that outcome acceptable rather than catastrophic. A site that degrades is a site that keeps working for the person on the worst phone and the weakest connection — which, on a university campus, is a lot of people.

13.6 Never trust the client

Your form validates carefully in the browser *and* the database re-validates everything. That is not redundancy. Anyone can bypass your JavaScript entirely and POST straight at the API. Browser validation is a courtesy to the person filling the form; the database constraint is the guarantee.

Generalise it: any check that matters must happen somewhere the user cannot reach.

13.7 Security is about permissions, not secrecy

Your Supabase key is published and that is correct. What protects the data is a rule saying anonymous visitors may insert and may do nothing else.

The general form: assume anything in the frontend is public, because it is. Grant the minimum permission that makes the feature work, and nothing more. Your database has no select policy, no update policy and no delete policy — not because they were forgotten, but because each would be an unnecessary permission.

13.8 Every dependency is a liability

Your site has zero and will run untouched for years. A typical beginner project has hundreds, each an opportunity for breakage, abandonment or a security advisory.

The question to ask before adding anything: what does this do that thirty lines of my own code could not? Sometimes the honest answer justifies it. Often it does not.

13.9 Version control is not backup — it is time travel

Git lets you see exactly what you changed before committing, undo a specific change from weeks ago, and recover a file you deleted. Your fifteen commits each carry a message explaining *why*, which is how you will understand your own decisions in a year.

Habits worth keeping: commit small and often; write messages that explain the reasoning, not the diff; never commit something you have not looked at with `git diff`.

13.10 Performance is mostly about what you do not send

Your page is around 200 KB before the video is conditionally attached. The videos are 97% of the total, which is exactly why they are gated behind three checks.

The lesson: the fastest asset is the one never downloaded. Before optimising code, look at what you are shipping at all. One unoptimised photograph can outweigh your entire codebase.

13.11 Accessibility is not an extra

Your ARIA tabs, keyboard navigation, form labels and reduced-motion support cost very little and were built in from the start. Retrofitting them would have been a rewrite.

The cheapest rules: use real `<button>` elements; give every image an `alt`; label every input; make sure everything is reachable by Tab. That is most of the value for very little effort.

13.12 Technical debt, and the honest kind

Debt is a shortcut you take deliberately, knowing you will pay interest. Your project has some and it is *documented*, which is what makes it debt rather than rot: `netlify.toml` left in place, the register-number rule left permissive, the placeholder content, the portal's single-instance state.

The distinction: a known shortcut with a note is manageable. An unknown one is a trap waiting for someone who has forgotten. Write the note.

13.13 Write down why, not what

Your code comments explain reasoning: why the video src is attached in JavaScript, why the scoring divides by a square root, why rAF is not used for the audio ramp. The code already says what it does; only a comment can say why.

`DESIGN.md` goes further and records mistakes that were made and corrected. That is unusual and genuinely valuable — it is what stops the same mistake being made twice.

What to learn next

Prioritised by what would actually make you capable of maintaining and extending *this* site. Not a general web-development curriculum — most of that you do not need.

Level 1 — Must understand now

Without these you are still dependent. Aim for a few weeks.

Topic	Why, for your site specifically	How to learn it
Browser DevTools	The highest-leverage skill on this list. Console, Elements, Network. Without it you are guessing; with it you are diagnosing.	Open your own site and inspect things for an hour. Break something deliberately and find it.
Git basics	<code>status</code> , <code>diff</code> , <code>add</code> , <code>commit</code> , <code>push</code> , <code>log</code> , <code>checkout</code> , <code>revert</code> . This is your deployment and your undo.	The first three chapters of the free Pro Git book. Then use it daily.
JavaScript objects and arrays	Editing <code>content.js</code> safely. Where commas go, why quotes matter, nesting.	MDN's "JavaScript first steps". You need the data half, not the whole language.
CSS selectors and specificity	The single trap that has bitten this project twice. Fixing anything visual requires it.	MDN on specificity, then practise in the Elements panel where the answer is visible.
Reading an error message	They name the file and the line. Beginners skim them.	Deliberately cause errors and read what you get.

Level 2 — Should learn soon

These let you add features rather than only maintain them. A couple of months.

Topic	Why	How
DOM manipulation	<code>createElement</code> , <code>textContent</code> , <code>append</code> , <code>querySelector</code> . This is what every renderer in <code>app.js</code> is doing.	Write a small renderer of your own — a "sponsors" section, say.
Events and listeners	How every interaction works. Including <i>delegation</i> , which <code>sound.js</code> uses.	Read <code>initTabs()</code> and <code>sound.js</code> line by line until both are obvious.
Flexbox and Grid	Every layout in your stylesheet uses one or the other.	Flexbox Froggy and Grid Garden — both free, both games, both about an hour.
Media queries	Section 20 of your stylesheet is where all phone fixes go.	Resize your browser and watch what changes.
HTTP basics	Status codes, GET vs POST, headers. Needed to debug anything involving Supabase.	MDN's HTTP overview, then read the Network tab on a real request.

SQL fundamentals	<code>select</code> , <code>insert</code> , <code>where</code> , and what a constraint is. You need this to read your own registrations.	SQLBolt — free, interactive, a few hours. Then query your own table.
-------------------------	--	--

Level 3 — Useful later

When the club's needs grow past what the site does today.

Topic	What it would unlock	Note
Async JavaScript	<code>fetch</code> , promises, <code>async/await</code> . Your form already uses all three at <code>join.js:199</code> .	Read that function until you can explain every line.
Supabase properly	Reading data back into the site — a live leaderboard instead of a hardcoded one.	Needs a careful read policy. Only expose what should be public.
A little Node.js	The concept of a backend. Enough to understand what you do not have.	Your <code>serve.js</code> is 33 lines of Node — read it, it is complete.
Serverless functions	The realistic path to "email me when someone registers" without running a server.	Supabase Edge Functions, or Cloudflare Workers.
Image and video optimisation	Keeping the site fast as real photos arrive. WebP, AVIF, responsive images.	Squoosh.app teaches this by letting you see the trade-off.
A framework	Only when you have a project that genuinely needs one.	You do not need one for this site. Adding React here would be strictly worse.

Level 4 — Advanced

Genuine specialisms. Pursue out of interest, not obligation.

- **GLSL and shader programming** — what `portal.js` is written in. The Book of Shaders is the standard introduction. Until then, treat that file as finished.
- **WebGL properly** — buffers, textures, the render pipeline. WebGL Fundamentals.
- **Web performance** — Core Web Vitals, the rendering pipeline, why animating layout properties is expensive.
- **Accessibility in depth** — the WCAG guidelines, and actually testing with a screen reader.
- **Video encoding** — ffmpeg filters, codecs, bitrate. Directly useful if the club keeps making cinematic assets.

A note on how to learn this

USE YOUR OWN SITE AS THE TEXTBOOK

You have a working, well-commented, real project that you care about. That is worth more than any tutorial, because every concept has something concrete to attach to.

The most effective exercise: **pick one function in `app.js` each week and read it until you can explain every line aloud.** There are about thirty. In under a year you would understand your entire codebase — which is a different and much better thing than having watched thirty hours of video.

Running this site without a subscription

What you have to manage, what each job actually requires, and which tools — free, paid, or none at all — are worth using for it. Written on the assumption that your Claude subscription ends and does not come back.

THE SHORT ANSWER

You can run this site indefinitely with no AI at all. Two scripts in `scripts/` do the work that judgement used to: one checks your content before you publish it, the other checks your database is not leaking. Neither needs an account, an internet connection, or a subscription.

AI is a convenience for the tedious parts, not a requirement for any of them.

15.1 Everything you must manage, by how often

This is the honest list. Most of it is content, and content needs no tooling beyond a text editor.

Job	How often	What it actually involves	Hardest part	Tool needed
Leaderboard & rosters	After events	Edit one array in <code>content.js</code> , run the checker, push.	Typing the data without breaking a comma.	None. Editor + checker.
Events & deadlines	Weekly in season	Add an entry. Past ones archive themselves; the countdown retargets itself.	Remembering <code>YYYY-MM-DD</code> .	None.
Blog posts	After events	Resize the photo, drop it in <code>assets/blog/</code> , add an entry.	Resizing. A phone photo is 4–8 MB and will wreck the page.	Squoosh.app (free, browser).
Chatbot answers	Monthly	Add keywords to an entry, or add an entry.	Thinking of the words people actually type.	None , but a chatbot is genuinely good at brainstorming keyword lists.
Reading registrations	Weekly in season	Supabase dashboard → Table Editor. Export CSV.	Nothing. It is a web page with a table.	None.
Backups	Monthly	<code>git push</code> covers code. Export the registrations CSV by hand.	Remembering. Put it in your calendar.	None.
Security check	Quarterly	<code>bash scripts/verify-supabase.sh</code>	Nothing. It prints pass or fail.	None.
Deploying	Every change	<code>git add -A && git commit -m "..."</code> && <code>git push</code>	Nothing, once memorised.	None.

Fixing a visual bug	Rarely	Inspect the element, find the losing rule, change a value.	CSS specificity. Genuinely fiddly.	Free chatbot helps a lot here.
Adding a feature	Rarely	New markup, new CSS, sometimes new JavaScript.	All of it, at first.	An agentic tool earns its money here, or nowhere.
Touching <code>portal.js</code>	Never	Do not. It works, it degrades gracefully, and it is a specialism.	—	—

15.2 The two scripts that replace judgement

These were written specifically so you would not need an AI for the two things that can actually hurt you: breaking the site, and leaking student data.

Before every commit

```
node scripts/check-content.js
```

Parses `content.js`. If you broke it, it prints the offending line with its neighbours and the three causes it is almost always one of. If you did not, it checks the things that fail *quietly* instead: dates in the wrong format, links missing `https://`, images that are not on disk, images heavy enough to hurt on mobile data, placeholder names still in the roster, FAQ entries with too few keywords, and keywords generic enough that one answer will start swallowing unrelated questions.

It also refuses a `service_role` key by decoding the token rather than trusting what the variable is called.

After any database change, and quarterly

```
bash scripts/verify-supabase.sh
```

Checks four things against your live project: an insert is accepted, a select comes back **empty**, an update is refused, a delete is refused. The select is the one that matters — your key is published in the page source by design, so if a select succeeds, anyone can download every student's phone number.

WHY THIS MATTERS MORE THAN ANY AI TOOL

An AI can write code for you. It cannot tell you whether your database is leaking right now, and it will confidently assure you things are fine without checking. These two scripts check. Run them and you do not need anyone's opinion.

15.3 The tool options, honestly compared

Option	Cost	What it does for this site	Verdict
--------	------	----------------------------	---------

Nothing Editor + the two scripts	Free	Everything in the top half of the table in 15.1 — which is 90% of the work. You edit <code>content.js</code> , the checker catches mistakes, you push.	Genuinely sufficient. Start here. Add a tool only when you hit something this cannot do, which will be rare.
VS Code	Free	Not AI. A proper editor: colours the syntax, underlines a missing comma <i>as you type</i> , matches brackets, has a built-in terminal so you never leave the window.	Install this first, before any AI tool. It prevents more mistakes per rupee than anything else on this list, and the price is zero.
Free ChatGPT / Claude.ai / Gemini	Free	Cannot touch your files. Very good at: formatting a spreadsheet into correct entries, explaining an error message, brainstorming FAQ keywords, explaining what a line of CSS does. You paste the result in yourself.	The best value on this list. Use with the prompts in <code>scripts/AI-PROMPT.md</code> . Covers most of what you lose.
GitHub Copilot	Free tier ~\$10/mo paid	Autocompletes inside VS Code. Helpful when writing new CSS or JavaScript. Free tier is limited but real, and free for students — check GitHub Student Pack with your RUAS email.	Worth the free tier if you are writing code. Not worth paying for if you are only editing content.
Cursor	Free tier \$20/mo paid	An editor with an agent that <i>can</i> edit your files directly — the closest substitute for what you have now. Free tier gives limited requests per month.	The real replacement, if you want one. The free tier is enough for occasional feature work. Only pay monthly if you are actively building.
Windsurf / Zed / Continue.dev	Free tiers	Same category as Cursor. Continue.dev can run a free local model on your own machine, so there is no subscription and no data leaves your laptop.	Worth knowing they exist if Cursor's free tier runs out.
Claude Pro / Claude Code	~\$20/mo	What you have now.	Worth resubscribing for a <i>burst</i> — a month when you are adding a real feature. Not worth holding through a quiet semester where you only update a roster.

What to actually do

1. **Install VS Code today**, while you still have help available. Free, and it is the single biggest reduction in how often you break something.
2. **Let the subscription lapse.** Run on the two scripts plus a free chatbot.
3. **If you hit something you cannot do** — a new feature, a bug you cannot locate — buy *one month* of Cursor or Claude, do the work, cancel. This is far cheaper than holding a subscription for a site that changes twice a semester.

THE RULE THAT KEEPS YOU INDEPENDENT WHICHEVER YOU CHOOSE

Never apply a change you cannot explain. If a tool hands you code and you cannot say why it works, ask it to explain before you use it. A site held together by changes you do not understand is a site you cannot maintain — you will have swapped one dependency for another, and the next one may not be as easy to ask.

15.4 What to watch out for

Not requested, but these are the things most likely to go wrong for you specifically, ranked by how likely they are to actually bite.

#	Risk	Why it is likely	What to do
1	Registrations collected before the database is verified	The form shows a success screen whether or not the data arrives anywhere. You would not find out at a club fair until afterwards.	Run <code>verify-supabase.sh</code> and submit one real test registration <i>before</i> sharing the link. Check the row appears in the dashboard.
2	Placeholder content going public	<code>PLAYER-01</code> and <code>Full Name</code> are still on the Players tab right now. The checker warns about this on every run.	One afternoon with <code>content.js</code> . This matters more than any code change in this manual.
3	The Supabase project is tied to a personal account	When you graduate, the club's registration data goes with you, or is lost.	Record which email owns it. Consider a club-owned account. Same for GitHub.
4	Free-tier database paused for inactivity	Supabase pauses free projects after about a week of no activity. The form then fails silently and submissions queue in visitors' browsers.	Open the dashboard occasionally. Export the CSV monthly so a pause is never a loss.
5	Huge unoptimised photos	The obvious next step is adding real event photos straight from a phone.	Resize to about 1600px first. The checker warns above 600 KB.
6	Video source files lost	Three <code>-source.mp4</code> files are gitignored and exist only on your laptop. A disk failure loses them permanently.	Copy them to cloud storage. This is a five-minute job you keep not doing.
7	Someone "tidies up" a load-bearing line	<code>[hidden] { display: none !important; }</code> looks like a stray one-liner. Deleting it makes the entire site unclickable.	It is documented in <code>ARCHITECTURE.md</code> . Read that before letting anyone else edit the stylesheet.
8	No error monitoring	If the site breaks for visitors, nothing tells you. You find out from a complaint, or never.	Open the live site on a real phone once a month and watch the console. That is the realistic substitute.
9	Personal data handled casually	You are collecting names, phone numbers and register numbers from students.	Do not share exports. Delete them when finished. Check whether RUAS requires a privacy notice before collecting at scale.

10	The GitHub account is a single point of failure	Lose access and you lose the site and every backup at once.	Two-factor authentication, recovery codes stored somewhere physical, and a second committee member with access.
----	--	---	---

Checklists

Reusable, and written to be used rather than admired. The first four apply to this site; the last applies to whatever you build next.

16.1 Pre-deployment checklist

Before every `git push`. Two minutes.

- Previewed locally (`node serve.js`) and the change looks right.
- **Console is clean** — no new red errors. (The join-form warning is expected until 11.14 is done.)
- Checked at phone width — resize the window to about 375px.
- Clicked through all four tabs; none are blank.
- If you touched the form: submitted once on *both* Member and Roster paths.
- If you touched the FAQ: hovered the launcher, clicked in, came back out.
- If you touched `content.js`: `node scripts/check-content.js` is clean.
- If you touched the database or its keys: `bash scripts/verify-supabase.sh` passes all four.
- `git diff` read — you can explain every changed line.
- No secrets, keys or personal data in the diff.
- Commit message says *why*, not just what.

16.2 Update checklist

For content changes, which is most of what you will do.

- Editing `content.js`, not a logic file — if not, be sure you need to.
- Commas between every entry; quotes closed; brackets balanced.
- Dates are `YYYY-MM-DD`.
- New images optimised (under about 400 KB) and given `alt` text.
- New links include `https://`.
- Reloaded the preview — the section actually changed.
- Nothing else broke: the site still renders everywhere.
- Run 16.1 and push.

16.3 Backup checklist

Monthly, ten minutes.

- `git push` — all local work is on GitHub.
- `git status` shows nothing uncommitted that matters.

- **Supabase data exported to CSV** and stored somewhere private.
- Video source files (`*-source.mp4`) exist in cloud storage — *these are not in git*.
- `_archive-v1/` copied somewhere, if you still want it.
- Original logo artwork backed up.
- Once a semester: `git bundle create edge-backup.bundle --all` , stored off-machine.
- Once a year: actually test a restore (8.4). An untested backup is a hope.

16.4 Security checklist

At launch, then quarterly.

- **RLS verified:** `bash scripts/verify-supabase.sh` passes all four checks. Not "probably" — run it.
- Only an *insert* policy exists on `registrations` . No select, update or delete policy.
- The **service role key** appears nowhere in the repository. Search for it: `git log -p | grep -i "service_role"`
- No passwords, tokens or personal data anywhere in git history.
- GitHub account has two-factor authentication on, with recovery codes stored physically.
- Still no `innerHTML` anywhere: `grep -rn "innerHTML" js/` returns nothing.
- `[hidden] { display: none !important; }` still present in `styles.css` .
- HTTPS enforced — the live URL shows a padlock.
- Exported registration data is stored privately and deleted when finished with.
- Anyone who left the committee no longer has repository access.

16.5 Future-project checklist

Before writing a line of your next site.

Decide first

- Who is this for, and on what device and connection will they use it?
- What is the one thing it must do well?
- Does it genuinely need a backend, or is static enough? Static is cheaper, faster and safer — start there and add only when forced.
- What happens to it when you graduate or lose interest?

Set up before building

- `git init` and a first commit **before** anything else.
- A `.gitignore` written before the first commit, not after — history is permanent.
- A README saying what it is and how to run it, written on day one.
- Content separated from code from the start. Retrofitting this is painful.
- Deployment working on day one, with a placeholder page. Deploying for the first time at the end is how launches get delayed.

While building

- Commit small, commit often, explain why.
- Test on a real phone regularly, not just a resized window.
- Keep a running note of decisions and their reasons.
- For every feature: what happens when it fails? Make that acceptable.
- Resist dependencies. Each one needs a justification.
- Real `<button>` elements, `alt` on images, labels on inputs — from the start.

Before launch

- Works on a phone, on a slow connection.
- Works with JavaScript slow or partially failed — does it degrade or collapse?
- No secrets in frontend code.
- Any collected personal data is protected by a server-side rule, not by obscurity.
- Backups configured and tested.
- Someone other than you can deploy it.

The EDGE cheat sheet

One page. Print it, or keep it open.

Deploy

```
git add -A && git commit -m "what changed" && git push
```

Live in about a minute. Then hard-refresh: **Ctrl+Shift+R**. Preview first with `node serve.js` → `http://localhost:8790`

Where things live

All content	<code>js/content.js</code> — 90% of your edits
Leaderboard / rosters	<code>content.js</code> → <code>players</code>
Events & deadlines	<code>content.js</code> → <code>updates</code>
Chatbot answers	<code>content.js</code> → <code>faq.entries</code>
Blog posts	<code>content.js</code> → <code>blog</code>
Social links, email	<code>content.js</code> → <code>club</code>
Supabase keys	<code>content.js</code> → <code>join.backend.supabase</code>
Intro video & rules	<code>assets/video/intro.mp4</code> · <code>js/intro.js</code>
Prompt templates	<code>scripts/AI-PROMPT.md</code>
Colours & fonts	<code>css/styles.css</code> section 1 (tokens)
Phone fixes	<code>css/styles.css</code> section 20
Tab list	<code>js/app.js:45</code>
Videos & logos	<code>assets/</code>

Protect these

- **GitHub account** `potatoesareathing` — two-factor on, recovery codes stored. Losing this loses the site.
- **Supabase project** `wrjhopokogleplbbxntl` — note which email owns it. The **service_role** key must never appear in any file, ever.
- **Video sources** `assets/video/*-source.mp4` — three files, gitignored, not backed up anywhere. Copy them somewhere today.
- **Registration exports** — real student data. Store privately, delete when done.

Backup

Code: `git push` (automatic). Registrations: Supabase → Table Editor → Export CSV, **monthly**. Full snapshot: `git bundle create edge-backup.bundle --all`, each semester.

When something breaks

1. **F12** → Console. Red error? It names the file and line.
2. Section empty? Syntax error in `content.js` — missing comma or unclosed quote.
3. Nothing clickable? An invisible element is covering the page. Check `[hidden] { display: none !important; }` is still in the stylesheet.
4. Network tab → reload. Red rows = missing files (404) or a failing service.
5. Looks wrong but no errors? Right-click → Inspect. Struck-through rules lost on specificity.
6. Pushed but unchanged? Hard-refresh, then check the commit is on GitHub.
7. Undo: `git checkout -- path/to/file` · already pushed: `git revert <hash>`

Rules that must stay true

- `content.js` loads **first**; `portal.js` before `app.js`.
- Never `innerHTML`. Always `textContent`.
- Never start an asset path with `/` — relative paths only.
- Never add a blanket `img[src*="edge-mark"]` CSS rule.
- Match selector weight when overriding CSS (`table.board-table` , not `.board-table`).
- Only animate `transform` and `opacity`.
- Never use `requestAnimationFrame` for something that must simply happen.
- Never deploy by uploading a folder. Deploy with `git push`.
- The logo is 398×333 and the wordmark 665×183. Neither is square.
- A solid white fill means *primary action* and nothing else.

Do this first

Run `supabase-schema.sql`, then `bash scripts/verify-supabase.sh`. The keys are in and the project is live, but until the table exists every registration is still written to the student's own browser and lost. The verifier's second check must come back empty — your key is published in the page source by design, so if a select succeeds, anyone can download every student's phone number.

Live: potatoesareathing.github.io/edge-website/ · **Repo:** github.com/potatoesareathing/edge-website · **Local preview:** `node serve.js` → localhost:8790

Written from the codebase at commit `1ca0a30`, 17 September 2026; revised 26 September 2026 for Supabase, the intro and the phone layout. Regenerate from `docs/parts/` — see `docs/README.md`.